



Mobilne systemy operacyjne: Wprowadzenie

Marek Kulawiak, Krzysztof Bruniecki



- Istnieją zasadnicze różnice pomiędzy tradycyjnymi komputerami a urządzeniami przenośnymi:

Komputer stacjonarny (PC, Mac)	Urządzenie mobilne (smartfon, smartwatch)
Dostosowany do specjalistycznych programów (grafika, projektowanie)	Nastawione na komunikację
Oferuje wysoką wydajność	Przede wszystkim energooszczędne
Głównie stałe źródło zasilania	Zasilanie bateryjne
Często w zestawie z klawiaturą i myszką	Wyposażone w niewielki ekran dotykowy i minimalną liczbę przycisków
Zajmuje dużo miejsca	Mieści się w (dużej) kieszeni

Laptop? Tablet?



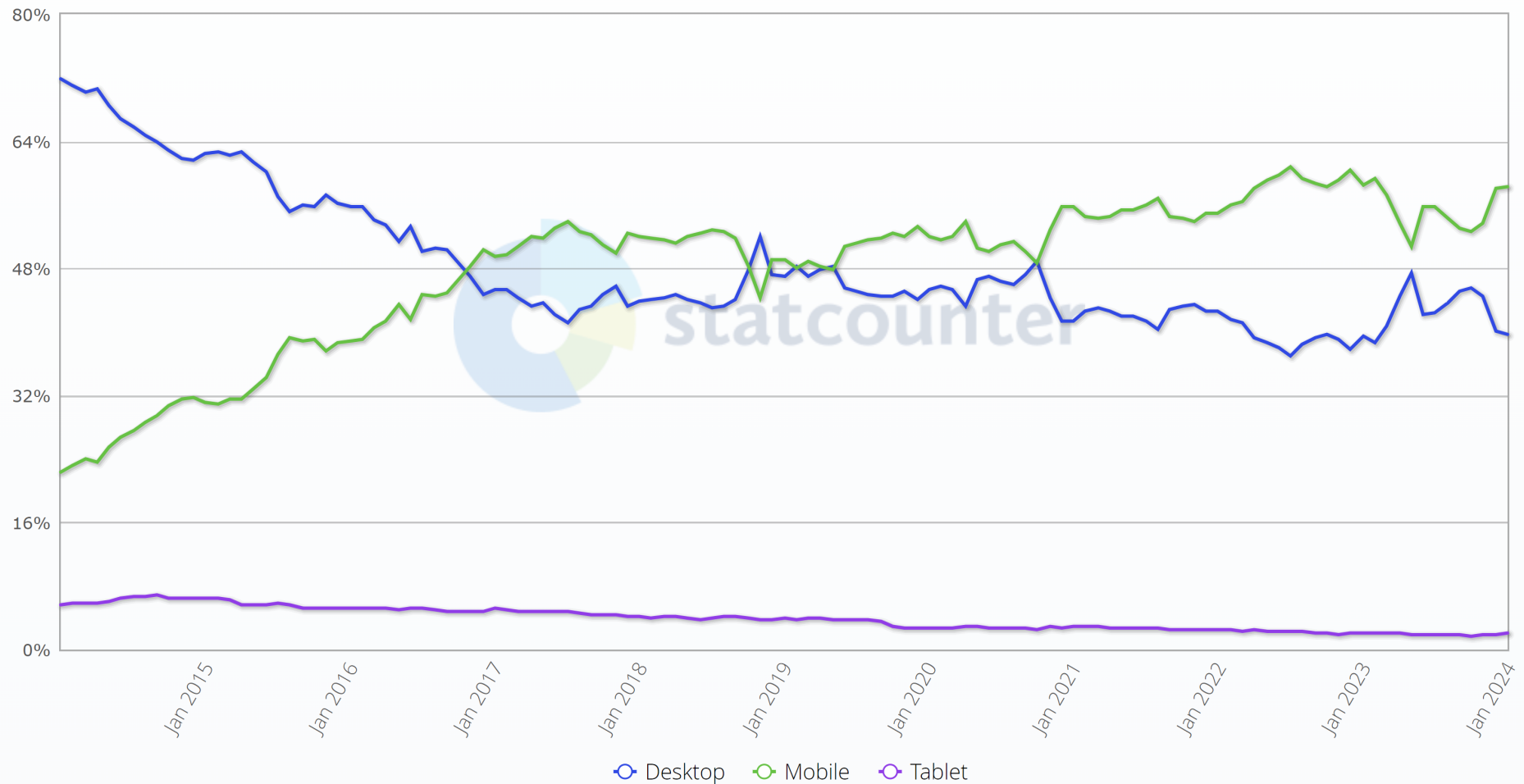
Pożądane cechy urządzeń mobilnych oraz ich systemów operacyjnych:

- Łączność bezprzewodowa
(Internet mobilny, Wi-Fi, Bluetooth, NFC, ...)
- Multimedia (audio/video, kamera, sensory, ...)
- Dostęp do lokalizacji (GNSS)
- Wygodny interfejs
(wiele pulpitów, widżety, powiadomienia, menu z aplikacjami)
- Wielozadaniowość
(współdziałanie telefonu, kamery, kalkulatora, ...)
- Zabezpieczenia (ekran blokady, uprawnienia, ...)
- Odporność na warunki pogodowe
(cecha fizyczna – OS nie zapewni wodoodporności)



Desktop vs Mobile vs Tablet Market Share Worldwide

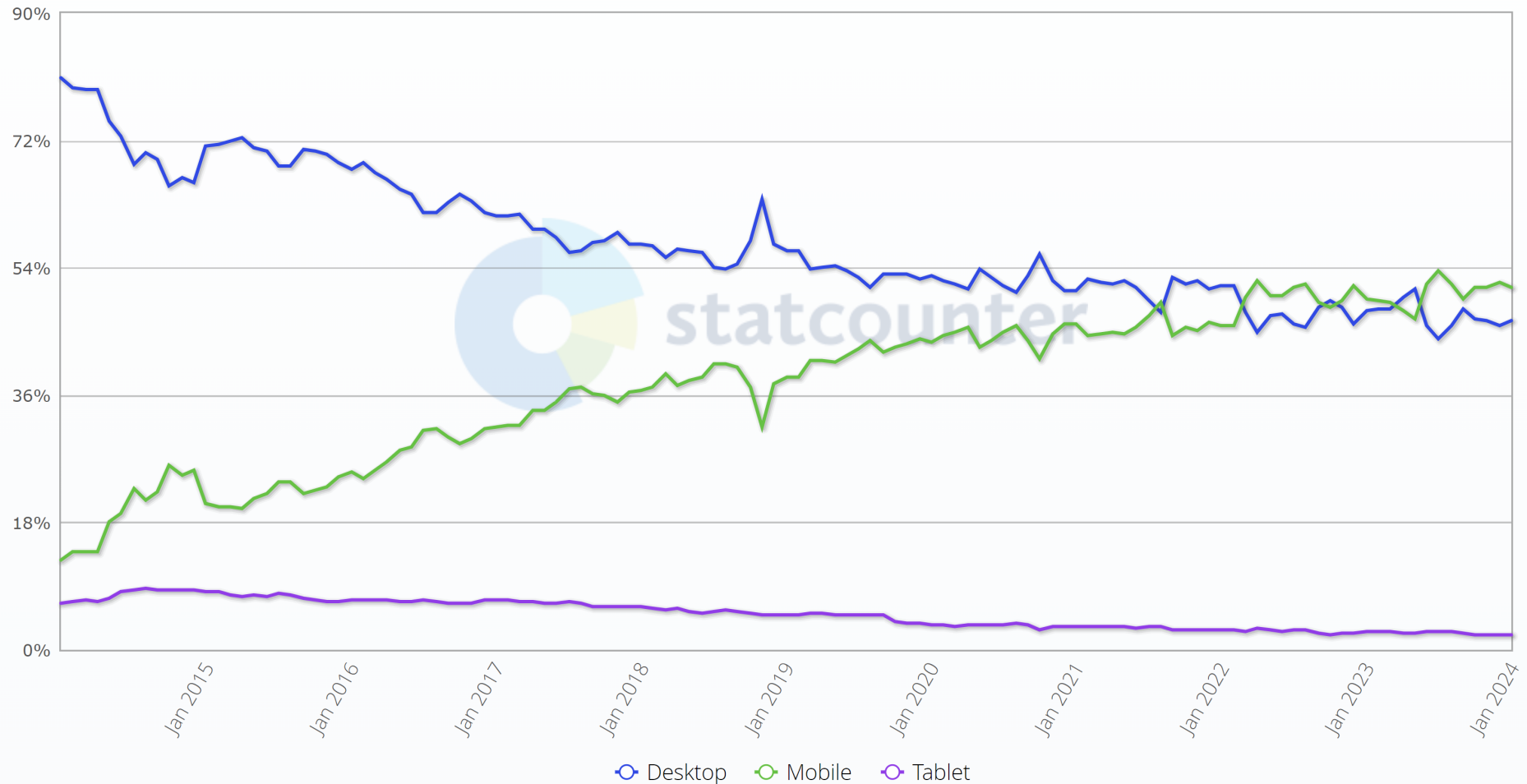
Jan 2014 - Jan 2024





Desktop vs Mobile vs Tablet Market Share Europe

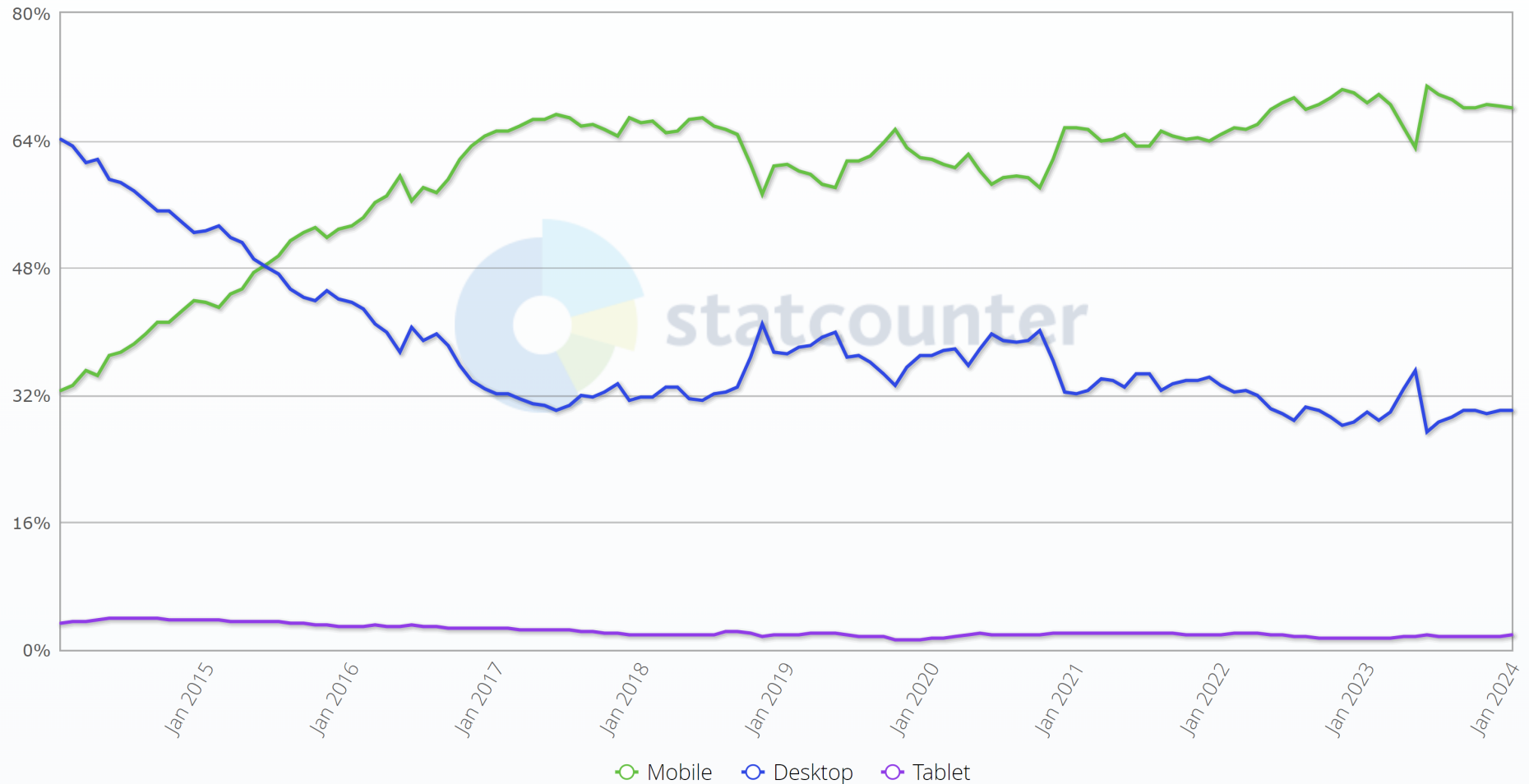
Jan 2014 - Jan 2024





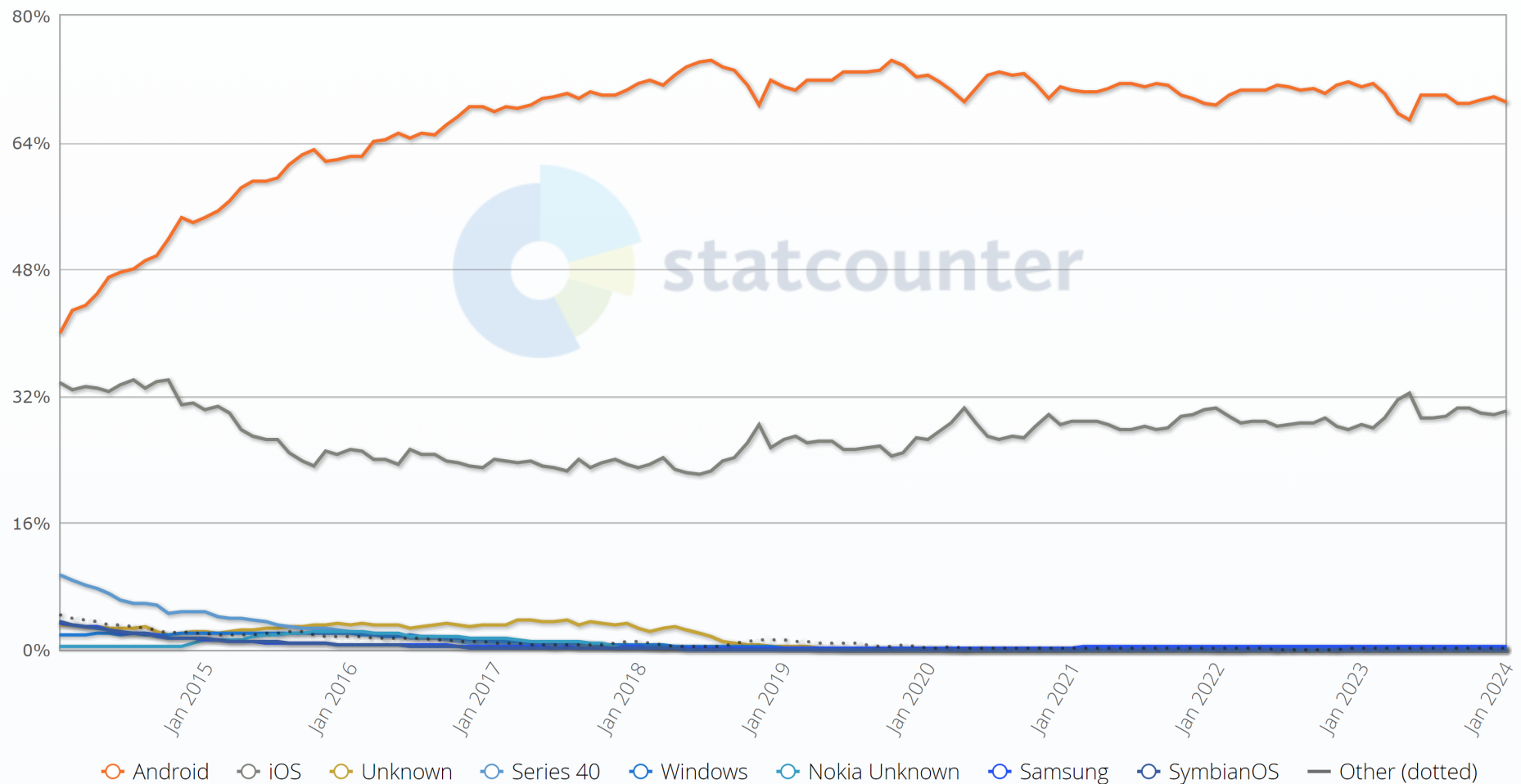
Desktop vs Mobile vs Tablet Market Share Asia

Jan 2014 - Jan 2024



Mobile & Tablet Operating System Market Share Worldwide

Jan 2014 - Jan 2024





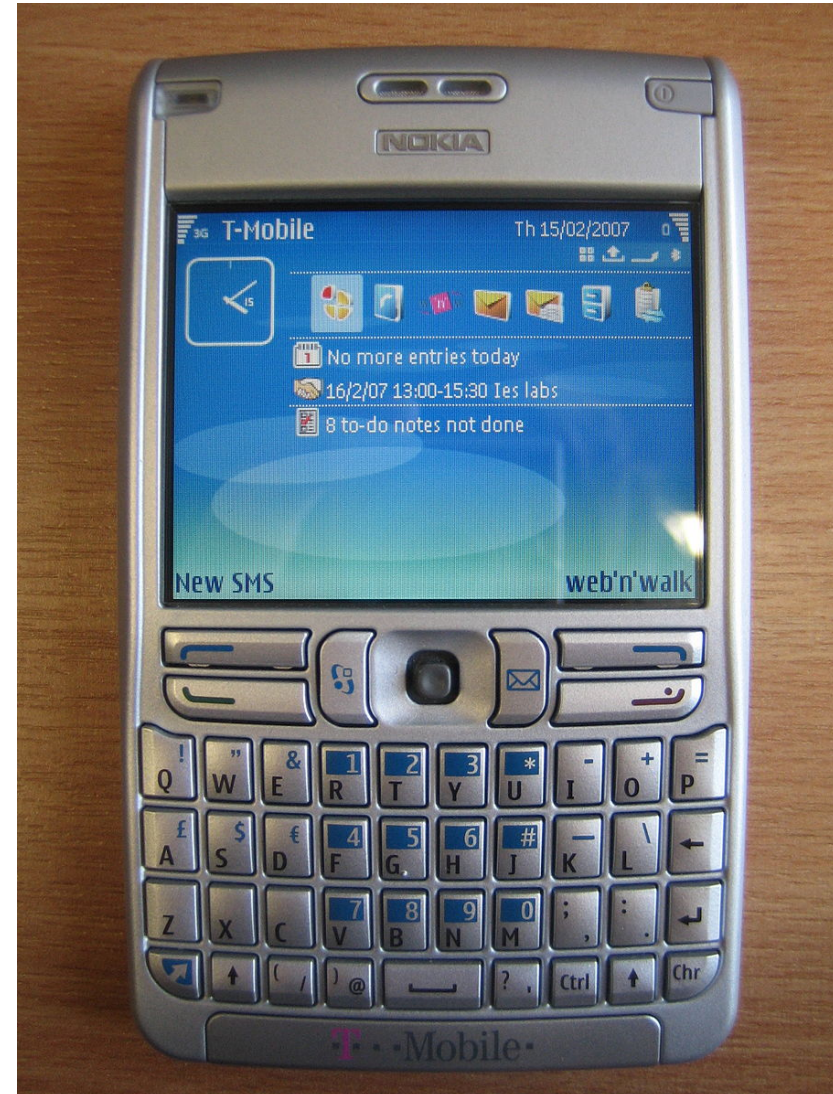
Rozwój urządzeń

- Urządzenia i usługi telefonii bezprzewodowej / mobilnej (telefony komórkowe, pagery) stały się dostępne i popularne w latach 90 XX wieku
- W latach 200X możliwości nowoczesnych telefonów komórkowych (smartfonów) zaczęły się zbliżać do możliwości komputerów osobistych
- Lata 20XX: duża popularność tabletów oraz nowego rodzaju urządzeń (smartwatch, smartband, telewizory)





- Premiera w 1998 roku
- Dla urządzeń z procesorem ARM
- Stosowany przez wiele firm (**Nokia**, Samsung, Motorola, LG, Sony Ericsson)
- Spadek popularności platformy na przełomie lat 2010/2011 (gdy Nokia wprowadziła urządzenia z Windows Phone)
- Rozwój aplikacji w Carbide.c++ / Python / QT / QML (JavaScript)





Historia: Windows na telefony

Firma Microsoft na przestrzeni lat stworzyła wiele mobilnych systemów operacyjnych:

- 2000: Pocket PC 2000, później przemianowany na Windows Mobile (w 2003 roku)
- 2010: Windows Phone 7, wyposażony w interfejs “kafelkowy” (Metro), przypominający interfejsy systemu Windows 8 oraz konsol Xbox 360 i Xbox One
- 2014: Windows Phone 8, następca WP7
- 2015: Windows 10 Mobile, reklamowany jako jedna z edycji Windows 10 (mimo że jest oparty na WP8), aktualnie już niewspierany



[https://en.wikipedia.org/wiki/
File:WP8.1_Start_Screen.png](https://en.wikipedia.org/wiki/File:WP8.1_Start_Screen.png)



- Urządzenia wprowadzone w 2000 roku przez firmę RIM (obecnie BlackBerry Limited)
- Ukierunkowane na klientów biznesowych i korporacyjnych
- BlackBerry utraciło popularność na rzecz systemów iOS i Android
- Kilka dostępnych SDK:
 - C/C++ SDK (niskopoziomowa)
 - WebWorks SDK (HTML, JavaScript)
 - Cascades (QML, JavaScript, C++)
- BlackBerry 10: możliwość przepakowania aplikacji z Androida (.apk → .bar)



https://en.wikipedia.org/wiki/File:RIM_BlackBerry_950.jpg



iOS:

- Wydany w 2007 roku
- Dostępny ekskluzywnie dla urządzeń firmy Apple (iPhone, iPad, itp.)

Android:

- System pierwotnie był rozwijany przez Android Inc., która w 2005 roku została wykupiona przez Google
- Od 2007 roku system jest rozwijany przez założone przez Google konsorcjum Open Handset Alliance (zrzeszające m.in. producentów smartfonów, podzespołów, oprogramowania, a także operatorów komórkowych)
- Sam system jest dostępny na zasadach open-source, ale większość urządzeń wykorzystuje także zamknięte oprogramowanie, takie jak Google Mobile Services



- Dodatkowe usługi i aplikacje od Google (m.in. Play Store, Youtube, Gmail i Google Maps), które są zazwyczaj preinstalowane na urządzeniach z Androidem
- Producent sprzętu musi wcześniej uzyskać od Google licencję (na szczęście darmową)
- Wraz z aplikacją Google Play Store instalowany jest także pakiet Google Play services, który umożliwia wykorzystanie usług Google we własnych aplikacjach



- Oficjalnym narzędziem do tworzenia oprogramowania jest Xcode
 - dostępne wyłącznie na systemie macOS
 - zawiera wsparcie dla wielu różnych języków programowania
 - na iOS najpopularniejszymi językami programowania są Swift i Objective-C



- Oficjalnym IDE jest Android Studio
 - dostępne na systemach Windows, macOS i różnych dystrybucjach Linuxa
 - oparte na IntelliJ IDEA
 - wykorzystuje Gradle jako narzędzie automatyzujące budowę oprogramowania
- Oficjalnie wspierane języki programowania:
 - Java (podzbiór Java SE oraz pakiety specyficzne dla Androida)
 - Kotlin (preferowany przez Google od czasu oskarżenia o naruszenie patentów Java)
 - C / C++ (za pomocą Android Native Development Kit)



Różne wersje Androida (maj 2024)

Android Platform/API Version Distribution



ANDROID PLATFORM VERSION	API LEVEL	CUMULATIVE DISTRIBUTION
4.4 KitKat	19	
5 Lollipop	21	99.7%
5.1 Lollipop	22	99.6%
6 Marshmallow	23	98.8%
7 Nougat	24	97.4%
7.1 Nougat	25	96.4%
8 Oreo	26	95.4%
8.1 Oreo	27	93.9%
9 Pie	28	89.6%
10 Q	29	81.2%
11 R	30	67.6%
12 S	31	48.6%
13 T	33	33.9%
14 U	34	13.0%

Last updated: May 1, 2024

U

New features

Ultra HDR for images
Zoom, Focus, Postview, and more in camera extensions
Lossless USB audio
Health Connect
Add custom actions
Custom meshes with vertex and fragment shaders

Behavior changes

Foreground service types are required
Enforcement of BLUETOOTH_CONNECT permission
JobScheduler reinforces callback and network behavior
Apps can kill only their own background processes
Schedule exact alarms are denied by default
Data safety information is more visible
Minimum installable target API level

Security and privacy

Credential Manager
Improvements for app stores
Detect when users take device screenshots
Secure full-screen Intent notifications
Restrictions to implicit and pending intents
Safer dynamic code loading restrictions on starting activities from the background
User consent required for each MediaProjection

<https://developer.android.com/about/versions/14>

OK

Cancel



- **Wear OS** (dawniej Android Wear) – okrojona wersja systemu Android, dostosowana dla urządzeń takich jak smartwatch i smartband; rozwijana przez Google, pierwsza wersja została udostępniona w 2014 roku [Android Studio]
- **watchOS** – zamknięty system przeznaczony dla urządzeń Apple Watch, stosowany od 2015 roku [Xcode]
- **Tizen** – system open-source rozwijany od 2012 roku przez firmę Samsung; wykorzystywany przez wiele rodzajów urządzeń, m.in. smartwatche, smart TV, kamery, odtwarzacze Blu-ray, smart home, również niektóre smartfony [Tizen Studio]
- Systemy rozwijane przez entuzjastów, np. AsteroidOS, Ubuntu Touch



Mobilne systemy operacyjne: Szybki wstęp do Android Studio

Marek Kulawiak



POLITECHNIKA
GDAŃSKA

Szablon projektu

New Project

×

Templates

Phone and Tablet

Wear OS

Television

Automotive

No Activity

Empty Activity

Gemini API Starter

Basic Views Activity

Bottom Navigation Views Activity

Empty Views Activity

Navigation Drawer Views Activity

Responsive Views Activity

Game Activity (C++)

Previous

Next

Cancel

Finish



Wstępna konfiguracja

 New Project ×

Empty Views Activity

Creates a new empty activity

Name

LayoutAndCompose

Package name

ksg.mso.layoutandcompose

Save location

D:\Work\Dydaktyka\MSO\LectureTests\LayoutAndCompose 

Language

Kotlin 

Minimum SDK

API 30 ("R"; Android 11.0) 

 Your app will run on approximately 67.6% of devices.
[Help me choose](#)

Build configuration language 

Kotlin DSL (build.gradle.kts) [Recommended] 

Previous

Next

Cancel

Finish



layoutAndCompose ▾ Version control ▾ Add Configuration... ▾ ▶ ⚙️ ⋮ 🛠️ 🔍 ⚙️ 👤 - □ ✕

activity_main.xml MainActivity.kt ✕

❗ Gradle project sync in progress...

Device Manager

```
1 package ksg.mso.layoutandcompose
2
3 import android.os.Bundle
4 import androidx.activity.enableEdgeToEdge
5 import androidx.appcompat.app.AppCompatActivity
6 import androidx.core.view.ViewCompat
7 import androidx.core.view.WindowInsetsCompat
8
9 class MainActivity : AppCompatActivity() {
10     override fun onCreate(savedInstanceState: Bundle?) {
11         super.onCreate(savedInstanceState)
12         enableEdgeToEdge()
13         setContentView(R.layout.activity_main)
14         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
15             val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
16             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
17             insets
18         }
19     }
20 }
```

LayoutAndCompose: Download <https://servic> 2 min
Download info

Download <https://repo.maven.apache.org/maven2/org/checkerframework/checker-qual/3.33.0/checker-qual-3.33.0.jar>
Download <https://repo.maven.apache.org/maven2/io/netty/netty-codec-http/4.1.93.Final/netty-codec-http-4.1.93.Final.jar>
Download <https://repo.maven.apache.org/maven2/io/netty/netty-handler/4.1.93.Final/netty-handler-4.1.93.Final.jar>

main > java > ksg > mso > layoutandcompose > MainActivity > onCreate Gradle: Building model... 18:10 LF UTF-8 4 spaces



Uruchomienie emulatora

The screenshot displays the Android Studio IDE with the following components:

- Top Bar:** Includes tabs for 'LayoutAndCompose' and 'Version control', and a dropdown for 'Small Phone API 30'.
- Left Panel:** Shows the 'Android' view with a tree structure containing 'app' and 'Gradle Scripts'.
- Editor:** Displays the source code of the app, with line numbers 1 through 20. The code is in Kotlin, and the current line is 18, which contains 'Hello World!'.
- Run Panel:** Shows the 'Run' button (a green play icon) and a dropdown for 'app'.
- Logcat Panel:** Displays the following log messages:

```
2025-02-19 12:55:13: Launching app on 'Small P
Starting: Intent { act=android.intent.action.M
Open logcat panel for emulator Small Phone API 30
Connected to process 31367 on device 'Small_Phone_API_30 [emulator-5554]'.
```
- Emulator Window:** A window titled 'Android Emulator - Small_Phone_API_30:5554' is open, showing a virtual phone screen with the time '1:04' and the text 'Hello World!'.
- Device Manager:** A panel on the right titled 'Device Manager' shows a list of virtual devices:

Name	API Level	Architecture
Small Phone API 30	30	x86
Wear OS Large Round API 28	28	x86



Przypisanie ID do kontrolki tekstowej

Small Phone API 30 app

activity_main.xml MainActivity.kt

Palette

Common

Ab TextView

Text

Buttons

Widgets

Layouts

Containers

Helpers

Google

Legacy

Component Tree

main

Ab textView "Hello World!"

activity_main.xml

Pixel 34 LayoutAndCompose

Attributes

Ab TextView textView

id textView

Declared Attributes

layout_width wrap_content

layout_height wrap_content

layout_constraint... parent

layout_constraint... parent

layout_constraint... parent

layout_constraint... parent

id textView

text Hello World!

Layout

Constraint Widget

Constraints

Start → StartOf parent (0dp)

End → EndOf parent (0dp)

androidx.constraintlayout.widget.ConstraintLayout > TextView



Dodanie przycisku

Small Phone API 30 app

activity_main.xml MainActivity.kt

Palette

- Common
 - Ab TextView
 - Button
 - ImageView
 - RecyclerView
 - FragmentC...
 - ScrollView
 - Switch
- Text
- Buttons
- Widgets
- Layouts
- Containers
- Helpers
- Google
- Legacy

Component Tree

- main
 - Ab TextView "Hello World!"
 - button "Button"

activity_main.xml

0dp 34 LayoutAndCompose

Attributes

Button button

id button

Declared Attributes

- layout_width wrap_content
- layout_height wrap_content
- layout_constr... parent
- layout_constr... parent
- layout_constr... parent
- layout_constr... parent
- layout_margin... 32dp
- id button
- text Button

Layout

Constraint Widget

32 0 0 50

Constraints

- Start → StartOf parent (0dp)
- End → EndOf parent (0dp)

androidx.constraintlayout.widget.ConstraintLayout > Button



Obsługa wciśnięcia przycisku

Small Phone API 30 app

Run 'app' Shift+F10

```
</> activity_main.xml MainActivity.kt
3  > import ...
10
11  </> class MainActivity : AppCompatActivity() {
12      private lateinit var mTextView: TextView
13      private lateinit var mButton: Button
14
15      override fun onCreate(savedInstanceState: Bundle?) {
16          super.onCreate(savedInstanceState)
17          enableEdgeToEdge()
18          setContentView(R.layout.activity_main)
19          ViewCompat.setOnApplyWindowInsetsListener(findView
20              val systemBars = insets.getInsets(WindowInset
21              v.setPadding(systemBars.left, systemBars.top,
22              insets
23          }
24
25      mTextView = findViewById(R.id.textView)
26      mButton = findViewById(R.id.button)
27      mButton.setOnClickListener {
28          mTextView.setText("Button clicked!")
29      }
30  }
```

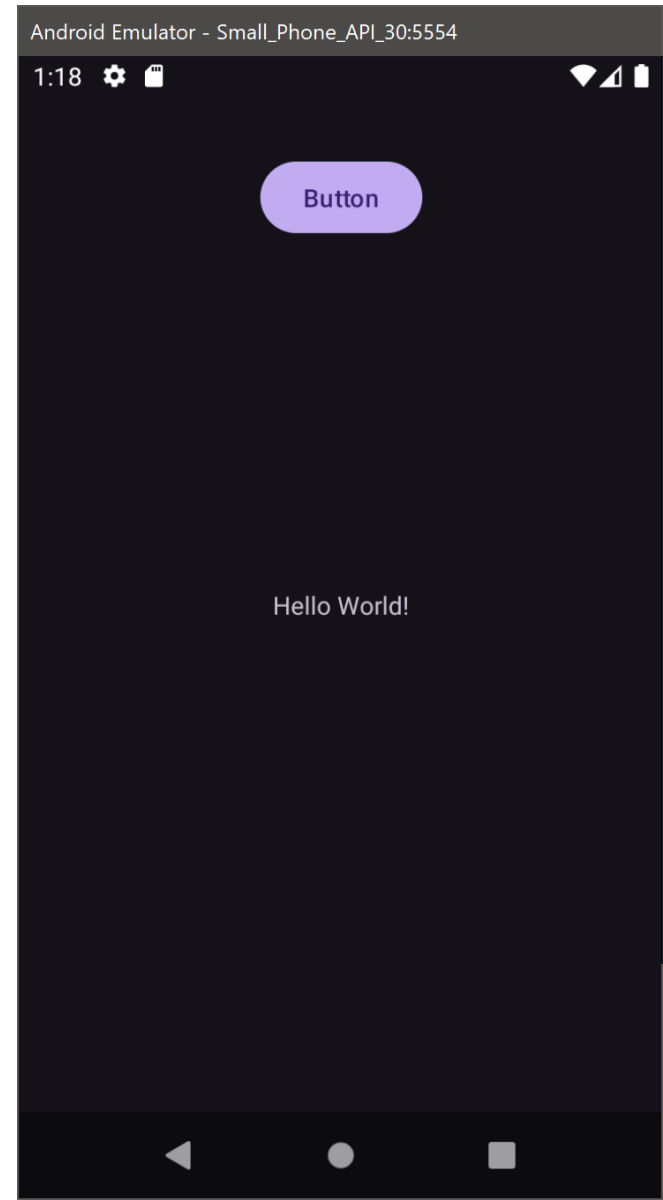
Device Manager

Name	...	...
Small Phone API 30 Android 11.0 ("R") x86	30 V...	⏏
Wear OS Large Round API 28 Android 9.0 ("Pie") x86	28 V...	▶



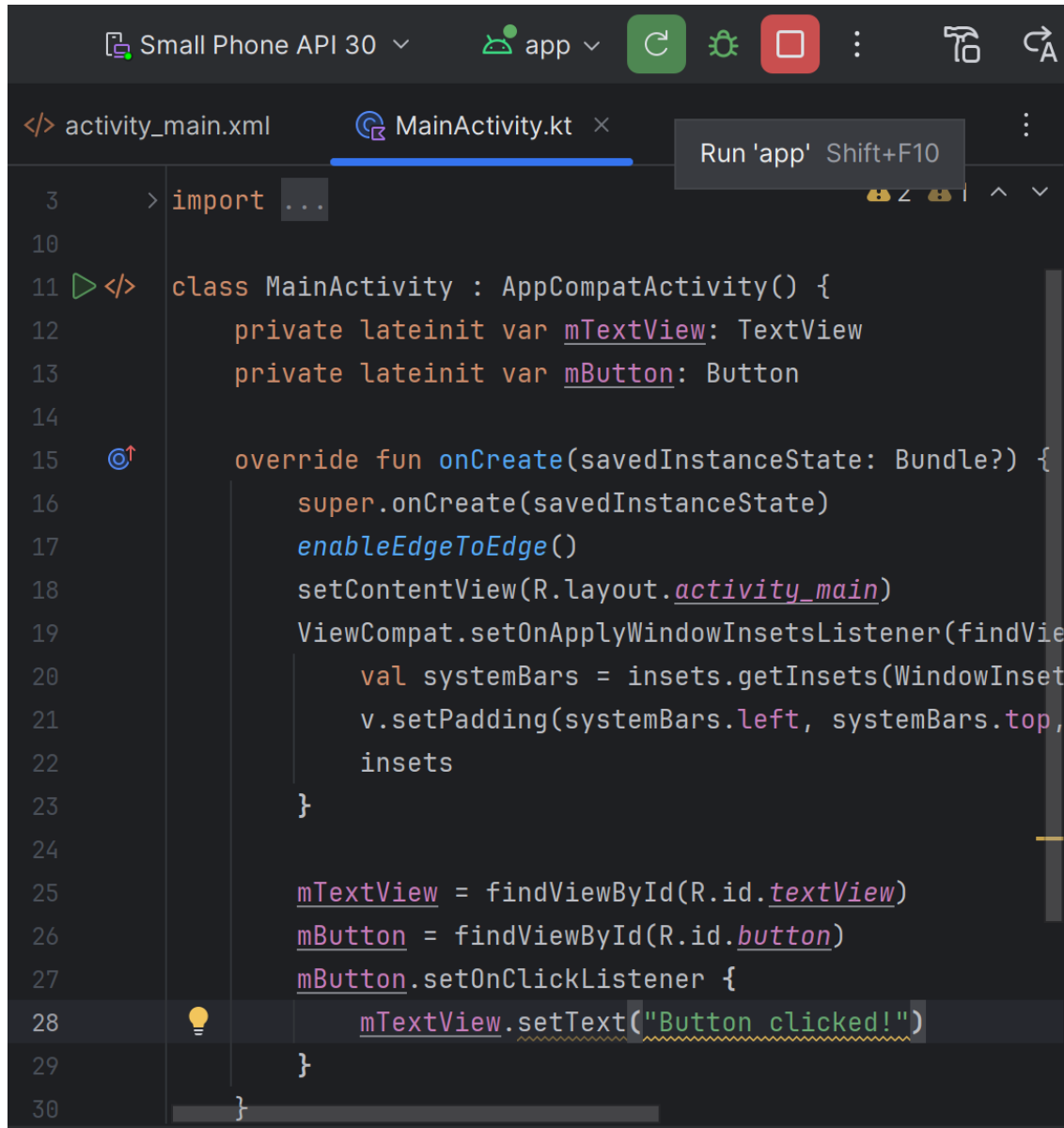
Obsługa wciśnięcia przycisku

```
</> activity_main.xml  MainActivity.kt  x  Run 'app' Shift+F10
3  > import ...
10
11  </> class MainActivity : AppCompatActivity() {
12      private lateinit var mTextView: TextView
13      private lateinit var mButton: Button
14
15      override fun onCreate(savedInstanceState: Bundle?) {
16          super.onCreate(savedInstanceState)
17          enableEdgeToEdge()
18          setContentView(R.layout.activity_main)
19          ViewCompat.setOnApplyWindowInsetsListener(findView
20              val systemBars = insets.getInsets(WindowInset
21              v.setPadding(systemBars.left, systemBars.top,
22              insets
23          }
24
25          mTextView = findViewById(R.id.textview)
26          mButton = findViewById(R.id.button)
27          mButton.setOnClickListener {
28              mTextView.setText("Button clicked!")
29          }
30      }
```

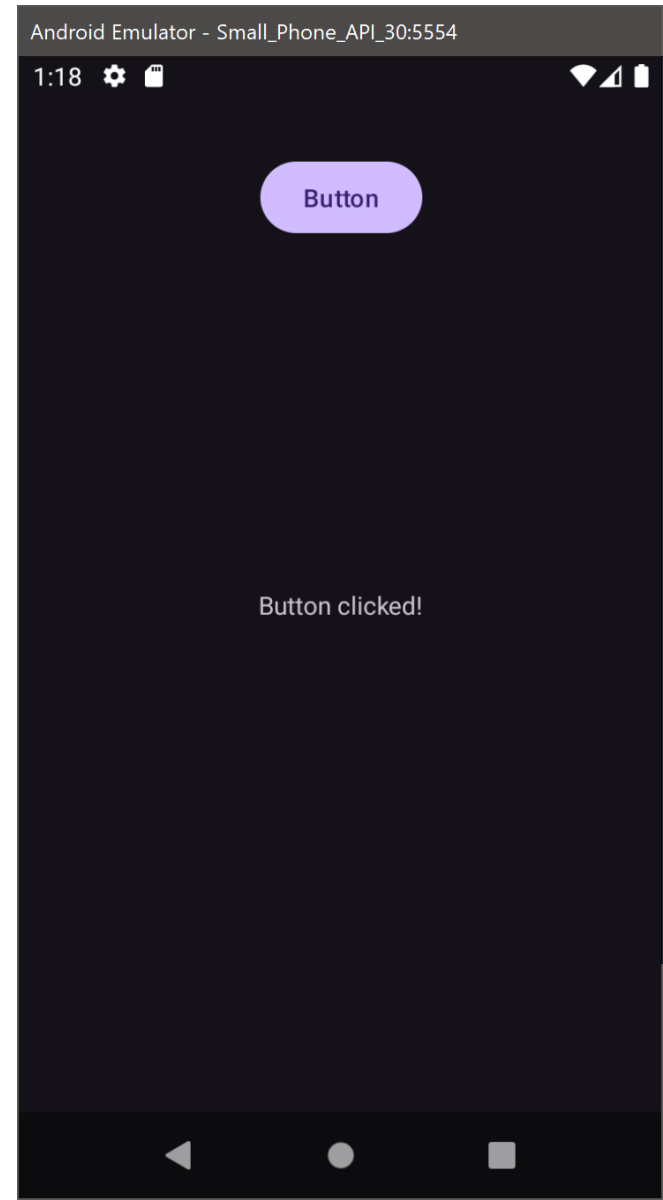




Obsługa wciśnięcia przycisku



```
</> activity_main.xml  MainActivity.kt  x  Run 'app' Shift+F10
3  > import ...
10
11  </> class MainActivity : AppCompatActivity() {
12      private lateinit var mTextView: TextView
13      private lateinit var mButton: Button
14
15      override fun onCreate(savedInstanceState: Bundle?) {
16          super.onCreate(savedInstanceState)
17          enableEdgeToEdge()
18          setContentView(R.layout.activity_main)
19          ViewCompat.setOnApplyWindowInsetsListener(findView
20              val systemBars = insets.getInsets(WindowInset
21              v.setPadding(systemBars.left, systemBars.top,
22              insets
23          }
24
25          mTextView = findViewById(R.id.text_view)
26          mButton = findViewById(R.id.button)
27          mButton.setOnClickListener {
28              mTextView.setText("Button clicked!")
29          }
30      }
```





Compose (gdy nie użyto szablonu Empty Views)

build.gradle.kts (:app) ×

```
android {  
    jvmTarget = "1.8"  
}  
buildFeatures {  
    compose = true  
}  
composeOptions {  
    kotlinCompilerExtensionVersion = "1.5.1"  
}  
}  
  
dependencies {  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    implementation(libs.androidx.activity)  
    implementation(libs.androidx.constraintlayout)  
    testImplementation(libs.junit)  
    implementation(libs.androidx.activity.compose)  
    implementation(platform(libs.androidx.compose.bom))  
    implementation(libs.androidx.ui)  
    implementation(libs.androidx.ui.graphics)  
    implementation(libs.androidx.ui.tooling.preview)  
    implementation(libs.androidx.material3)  
    androidTestImplementation(platform(libs.androidx.compose.bom))  
    androidTestImplementation(libs.androidx.test.espresso.core)  
    androidTestImplementation(libs.androidx.test.junit4)  
}
```

libs.versions.toml ×

```
1 [versions]  
2   activityCompose = "1.10.0"  
3   agp = "8.5.2"  
4   composeBom = "2025.02.00"  
5   kotlin = "1.9.0"  
6   coreKtx = "1.15.0"  
7   junit = "4.13.2"  
8   junitVersion = "1.2.1"  
9   espressoCore = "3.6.1"  
10  appcompat = "1.7.0"  
11  material = "1.12.0"  
12  activity = "1.10.0"  
13  constraintlayout = "2.2.0"  
14  
15 [libraries]  
16  androidx-activity-compose = { module = "androidx.activity:activity-compose", version.ref = "activityCompose"  
17  androidx-compose-bom = { module = "androidx.compose:compose-bom", version.ref = "composeBom"  
18  androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx"  
19  androidx-material3 = { module = "androidx.compose.material3:material3", version.ref = "material3"  
20  androidx-ui = { module = "androidx.compose.ui:ui", version.ref = "ui"  
21  androidx-ui-graphics = { module = "androidx.compose.ui:ui-graphics", version.ref = "ui-graphics"  
22  androidx-ui-test-junit4 = { module = "androidx.compose.ui:ui-test-junit4", version.ref = "ui-test-junit4"  
23  androidx-ui-test-manifest = { module = "androidx.compose.ui:ui-test-manifest", version.ref = "ui-test-manifest"  
24  androidx-ui-tooling = { module = "androidx.compose.ui:ui-tooling", version.ref = "ui-tooling"  
25  androidx-ui-tooling-preview = { module = "androidx.compose.ui:ui-tooling-preview", version.ref = "ui-tooling-preview"  
26  junit = { group = "junit", name = "junit", version.ref = "junit"  
27  androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junit"  
28  androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore"  
29  androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat"  
30  material = { group = "com.google.android.material", name = "material", version.ref = "material"  
31  androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity"  
32  androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout"  
33  
34 [plugins]  
35  android-application = { id = "com.android.application", version.ref = "agp"  
36  android-library = { id = "com.android.library", version.ref = "agp"  
37  kotlin-android-gradle-plugin = { id = "org.jetbrains.kotlin.android", version.ref = "kotlin"  
38  kotlin-jvm = { id = "org.jetbrains.kotlin.jvm", version.ref = "kotlin"  
39  kotlin-multiplatform = { id = "org.jetbrains.kotlin.multiplatform", version.ref = "kotlin"  
40  kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin"  
41  kotlin-test = { id = "org.jetbrains.kotlin.test", version.ref = "kotlin"  
42  kotlin-test-junit4 = { id = "org.jetbrains.kotlin.test.junit4", version.ref = "kotlin"  
43  kotlin-test-junit5 = { id = "org.jetbrains.kotlin.test.junit5", version.ref = "kotlin"  
44  kotlin-test-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit4", version.ref = "kotlin"  
45  kotlin-test-junit5-junit5 = { id = "org.jetbrains.kotlin.test.junit5.junit5", version.ref = "kotlin"  
46  kotlin-test-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit4", version.ref = "kotlin"  
47  kotlin-test-junit5-junit5-junit5 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5", version.ref = "kotlin"  
48  kotlin-test-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
49  kotlin-test-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
50  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
51  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
52  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
53  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
54  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
55  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
56  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
57  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
58  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
59  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
60  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
61  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
62  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
63  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
64  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
65  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
66  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
67  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit4", version.ref = "kotlin"  
68  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5-junit5-junit4", version.ref = "kotlin"  
69  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5.junit5.junit5.junit5.junit5-junit5-junit5-junit4", version.ref = "kotlin"  
70  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5.junit5.junit5-junit5-junit5-junit4", version.ref = "kotlin"  
71  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5.junit5-junit5-junit5-junit4", version.ref = "kotlin"  
72  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.junit5-junit5-junit5-junit4", version.ref = "kotlin"  
73  kotlin-test-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit5-junit4 = { id = "org.jetbrains.kotlin.test.ju
```



Compose (gdzie nie użyto szablonu Empty Views)

```
29 compileOptions {
30     sourceCompatibility = JavaVersion.VERSION_1_8
31     targetCompatibility = JavaVersion.VERSION_1_8
32 }
```

New

Add C++ to Module

Cut Ctrl+X

Copy Ctrl+C

Copy Path/Reference...

Paste Ctrl+V

Find Usages Alt+F7

Find in Files... Ctrl+Shift+F

Replace in Files... Ctrl+Shift+R

Analyze

Refactor

Bookmarks

Reformat Code Ctrl+Alt+L

Optimize Imports Ctrl+Alt+O

Delete... Delete

Open In

Local History

Repair IDE on File

Reload from Disk

Compare With... Ctrl+D

Mark Directory as

Java Class

Kotlin Class/File

Android Resource File

Android Resource Directory

Sample Data Directory

File

Scratch File Ctrl+Alt+Shift+Insert

Package

Image Asset

Vector Asset

GLSL Shader

CMakeLists.txt

Activity

Fragment

Folder

Service

UiComponent

Automotive

XML

Wear

TV

AIDL

Widget

Google

Compose

Other

Resource Bundle

EditorConfig File

Version Catalog

Gallery...

Android TV

Basic View

Bottom Na

Empty View

Fragment

Fullscreen

Login Views Activity

Navigation Drawer Views Activity

Primary/Detail Views Flow

Responsive Views Activity

Scrolling Views Activity

Settings Views Activity

Tabbed Views Activity

New Android Activity

Empty Activity

Basic Views Activity

Bottom Navigation Views Activity

Empty Views Activity

Fullscreen Views Activity

Google AdMob Ads Views Activity

Google Maps Views Activity

Google Pay Views Activity

New Android Activity

Empty Activity

Create a new empty activity with Jetpack Compose

Activity Name

MainActivity2

Package name

ksg.mso.layoutandcompose

☒ Launcher Activity



Uruchomienie szablonu Compose

```
15
16 <?> class MainActivity2 : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         enableEdgeToEdge()
20         setContent {
21             LayoutAndComposeTheme {
22                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
23                     Greeting(
24                         name = "Android",
25                         modifier = Modifier.padding(innerPadding)
26                     )
27                 }
28             }
29         }
30     }
31 }
32
33 @Composable
34 fun Greeting(name: String, modifier: Modifier = Modifier) {
35     Text(
36         text = "Hello $name!",
37         modifier = modifier
38     )
39 }
40
41 @Preview(showBackground = true)
42 @Composable
43 fun GreetingPreview() {
44     LayoutAndComposeTheme {
45         Greeting(name = "Android")
46     }
47 }
```

Android Emulator - Small_Phone_API_30:5554

2:42

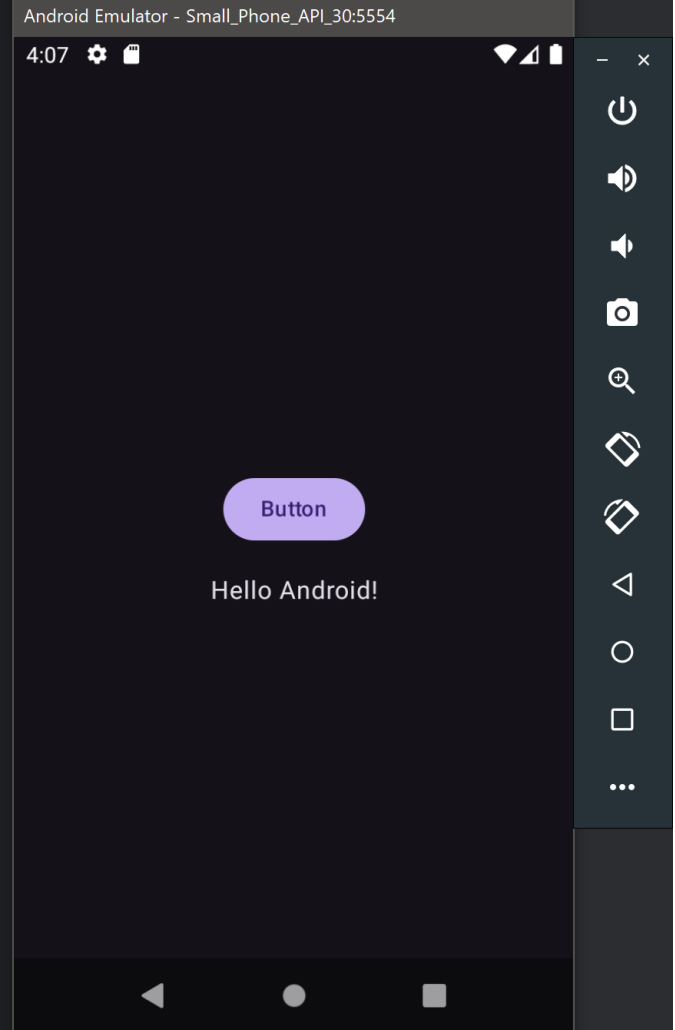
Hello Android!





Obsługa wciśnięcia przycisku (Compose)

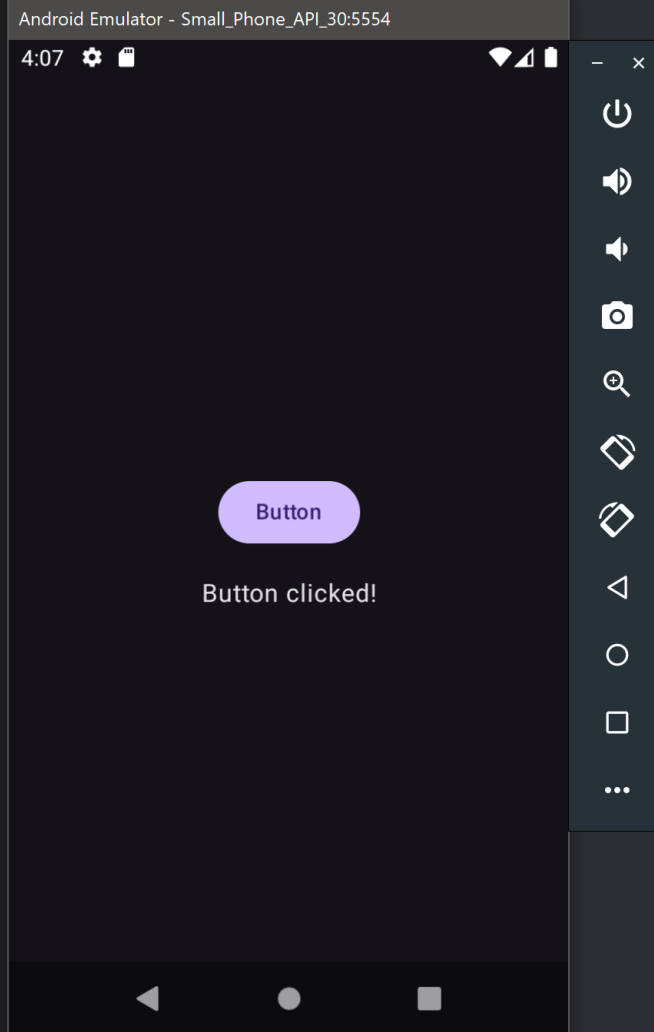
```
25     LayoutAndComposeTheme {
26         Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
27             Greeting(
28                 name = "Android",
29                 modifier = Modifier.fillMaxSize().padding(innerPadding)
30             )
31         }
32     }
33 }
34
35 }
36
37 @Composable
38 fun Greeting(name: String, modifier: Modifier = Modifier) {
39     var text by remember { mutableStateOf( value: "Hello $name!") }
40
41     Column(
42         modifier = modifier,
43         verticalArrangement = Arrangement.Center,
44         horizontalAlignment = Alignment.CenterHorizontally
45     ) {
46         Button(
47             modifier = Modifier.padding(top = 32.dp),
48             onClick = {
49                 text = "Button clicked!"
50             } {
51                 Text( text: "Button")
52             }
53
54         Text(
55             text = text,
56             modifier = Modifier.padding(top = 16.dp)
57         )
58     }
59 }
```





Obsługa wciśnięcia przycisku (Compose)

```
25         LayoutAndComposeTheme {
26             Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
27                 Greeting(
28                     name = "Android",
29                     modifier = Modifier.fillMaxSize().padding(innerPadding)
30                 )
31             }
32         }
33     }
34 }
35
36
37 @Composable
38 fun Greeting(name: String, modifier: Modifier = Modifier) {
39     var text by remember { mutableStateOf( value: "Hello $name!") }
40
41     Column(
42         modifier = modifier,
43         verticalArrangement = Arrangement.Center,
44         horizontalAlignment = Alignment.CenterHorizontally
45     ) {
46         Button(
47             modifier = Modifier.padding(top = 32.dp),
48             onClick = {
49                 text = "Button clicked!"
50             } {
51                 Text( text: "Button")
52             }
53
54         Text(
55             text = text,
56             modifier = Modifier.padding(top = 16.dp)
57         )
58     }
59 }
```





Poprawienie ustawień projektu

Android ▾

▽ app

- manifests
 - AndroidManifest.xml
- kotlin+java
 - ksg.mso.layoutandcompose
 - ui.theme
 - MainActivity
 - MainActivity2.kt
 - ksg.mso.layoutandcompose (androidTest)
 - ksg.mso.layoutandcompose (test)
 - res
 - res (generated)
- Gradle Scripts
 - build.gradle.kts (Project: LayoutAndCompose)
 - build.gradle.kts (Module :app)
 - proguard-rules.pro (ProGuard Rules for ":app")
 - gradle.properties (Project Properties)
 - gradle-wrapper.properties (Gradle Version)
 - libs.versions.toml (Version Catalog)
 - local.properties (SDK Location)
 - settings.gradle.kts (Project Settings)

MainActivity.kt MainActivity2.kt AndroidManifest.xml × libs.versions.toml

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <application
6         android:allowBackup="true"
7         android:dataExtractionRules="@xml/data_extraction_rules"
8         android:fullBackupContent="@xml/backup_rules"
9         android:icon="@mipmap/ic_launcher"
10        android:label="LayoutAndCompose"
11        android:roundIcon="@mipmap/ic_launcher_round"
12        android:supportsRtl="true"
13        android:theme="@style/Theme.LayoutAndCompose"
14        tools:targetApi="31">
15        <activity
16            android:name=".MainActivity2"
17            android:exported="true"
18            android:label="MainActivity2"
19            android:theme="@style/Theme.LayoutAndCompose">
20            <intent-filter>
21                <action android:name="android.intent.action.MAIN" />
22                <category android:name="android.intent.category.LAUNCHER" />
23            </intent-filter>
24        </activity>
25        <activity
26            android:name=".MainActivity"
27            android:exported="true">
28        </activity>
29    </application>
30
31 </manifest>
```




**Mobilne systemy operacyjne:
Podstawy programowania aplikacji w systemie Android**

Marek Kulawiak, Krzysztof Bruniecki

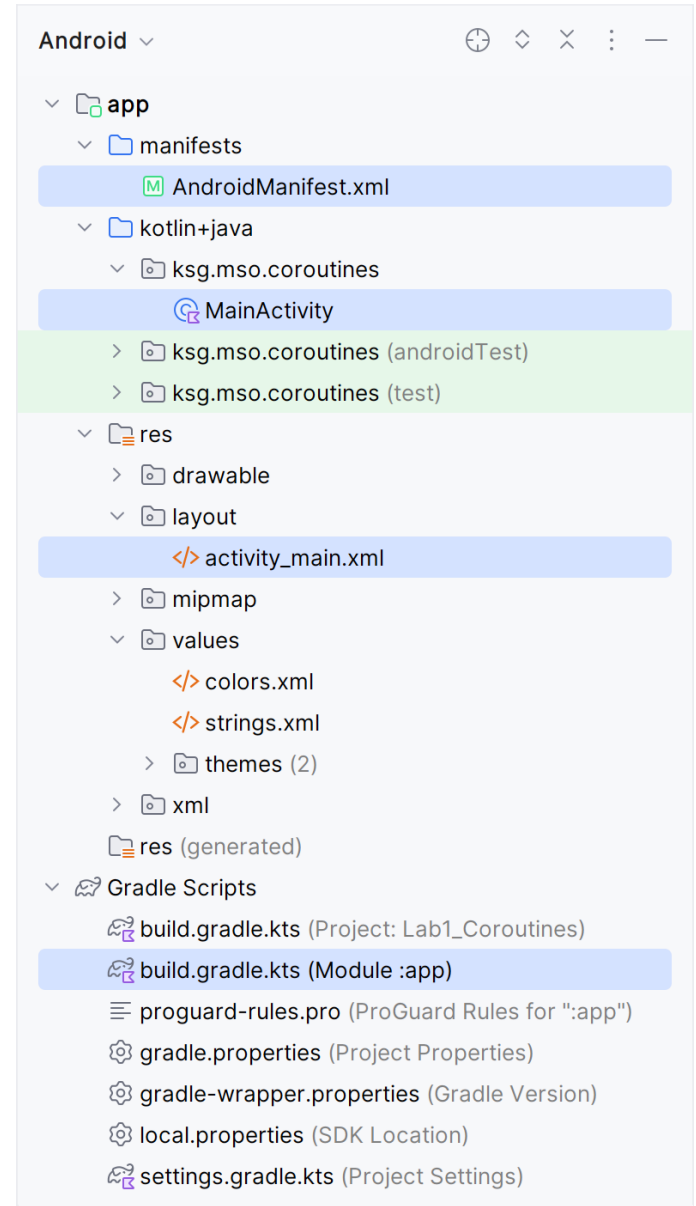


- Środowisko deweloperskie i potrzebne biblioteki
- Emulator oparty na QEMU
- Android SDK Platform Tools – zestaw pomocniczych narzędzi do komunikacji z urządzeniem:
 - **Android Debug Bridge (adb)** – program typu klient-serwer, umożliwiający dostęp do powłoki Unix; składa się z usługi (daemon) uruchamianej na urządzeniu oraz klienta i serwera uruchamianych na komputerze
 - **fastboot** – narzędzie do komunikacji z bootloaderem
 - ~~systrace~~ – ~~narzędzie do analizowania informacji o procesach pod kątem wydajności~~ (zamiast tego jest Studio Profiler i inne narzędzia)



Szkielet projektu w Android Studio

- Najprostszy projekt zawiera:
 - manifest opisujący najważniejsze komponenty aplikacji
 - aktywność definiującą logikę dla głównego okna aplikacji
 - układ graficzny dla tego okna (jeśli nie użyto Compose)
 - zestaw plików Gradle opisujących sposób budowania aplikacji





- Narzędzie automatyzujące budowę oprogramowania
- Do ważniejszych plików Gradle należą:
 - **settings.gradle.kts**, zawierający listę modułów
 - **build.gradle.kts** zamieszczony w głównym katalogu projektu, definiuje podstawowe ustawienia dla całego projektu
 - osobny plik **build.gradle.kts** dla każdego modułu w projekcie
 - **libs.versions.toml**, opisujący dokładne wersje bibliotek i wtyczek



build.gradle.kts dla modułu

```
android {  
    namespace = "ksg.mam.test"  
    compileSdk = 34 // minimalna wersja SDK wymagana do zbudowania projektu  
  
    defaultConfig {  
        applicationId = "ksg.mam.test"  
        minSdk = 30 // minimalna wspierana wersja API (Android 11)  
        targetSdk = 34 // najnowsza wersja API na której przetestowano apkę (A14)  
        versionCode = 1  
        versionName = "1.0"  
  
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"  
    }  
    (...)  
}  
  
dependencies {  
    // odwołania do zależności opisanych w libs.versions.toml:  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.appcompat)  
    implementation(libs.material)  
    implementation(libs.androidx.activity)  
    implementation(libs.androidx.constraintlayout)  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.androidx.junit)  
    androidTestImplementation(libs.androidx.espresso.core)  
}
```



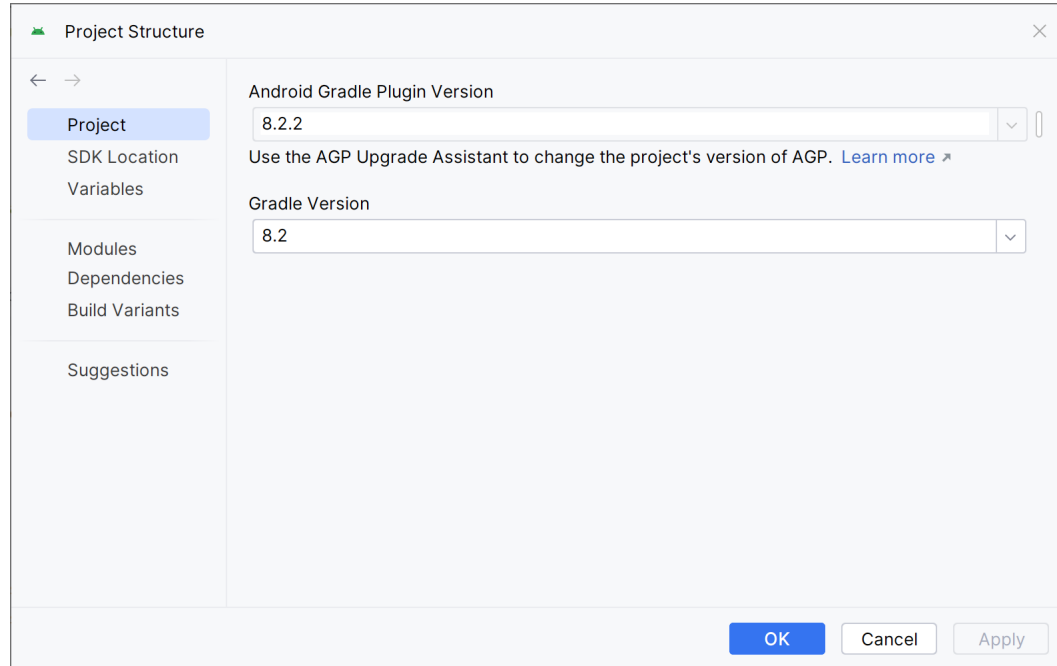
Poziomy API dla różnych wersji systemu

Codename	Version	API level/NDK release
Android14	14	API level 34
Android13	13	API level 33
Android12L	12	API level 32
Android12	12	API level 31
Android11	11	API level 30
Android10	10	API level 29
Pie	9	API level 28
Oreo	8.1.0	API level 27
Oreo	8.0.0	API level 26
Nougat	7.1	API level 25
Nougat	7.0	API level 24
Marshmallow	6.0	API level 23
Lollipop	5.1	API level 22
Lollipop	5.0	API level 21



Android Studio – konfiguracja projektu

- Wersja Gradle i wtyczki do niego
- Ścieżki do Android SDK, NDK i JDK
- Warianty budowania (debug, release)
- Warianty aplikacji, tzw. “flavours” (np. darmowy i płatny)
- Zależności





AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Test"
        tools:targetApi="31">
        <!-- Informacje o wszystkich aktywnościach w aplikacji -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <!-- Możliwość uruchomienia z poziomu pulpitu (App Drawer) -->
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```



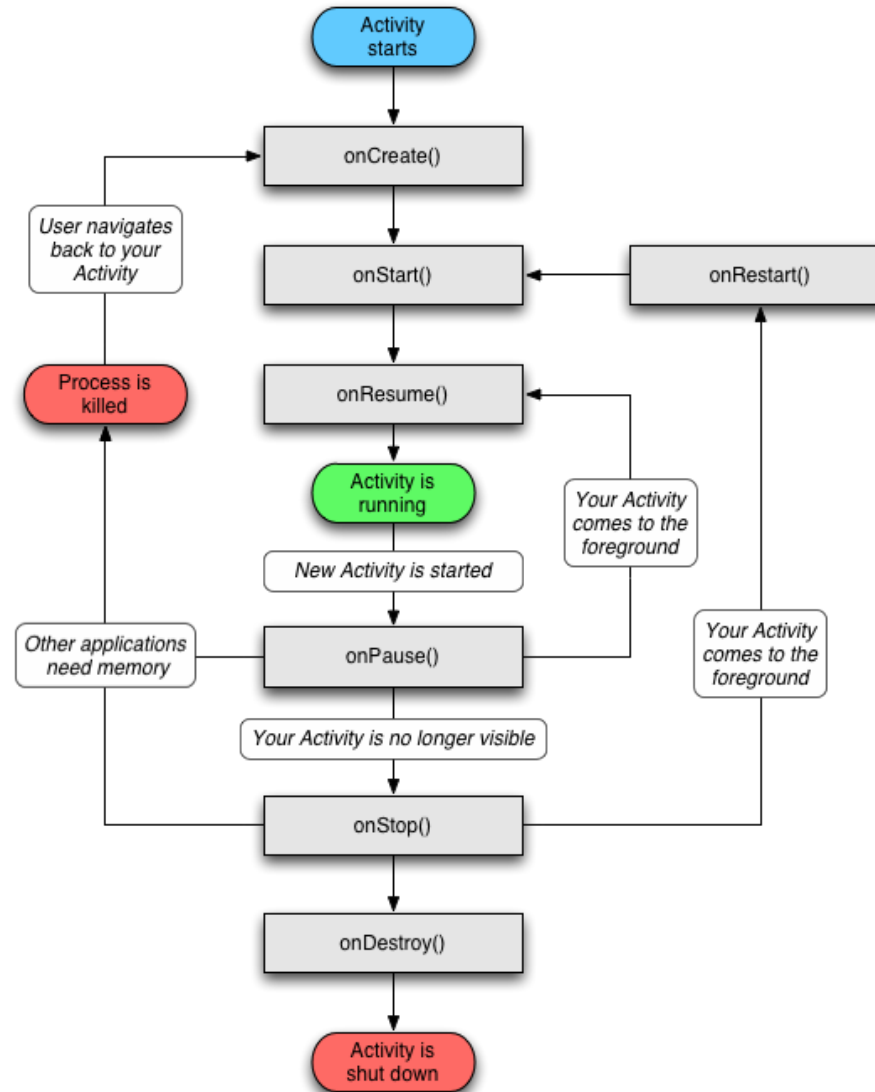

MainActivity – główna klasa aplikacji

```
class MainActivity : AppCompatActivity()  
{  
    override fun onCreate(savedInstanceState: Bundle?)  
    {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

- Klasa **R** identyfikuje wszystkie zasoby w projekcie
 - od czasu Android Studio 3.6 wtyczka Gradle generuje zawartość klasy **R** od razu w postaci kodu bajtowego



Cykl życia aktywności

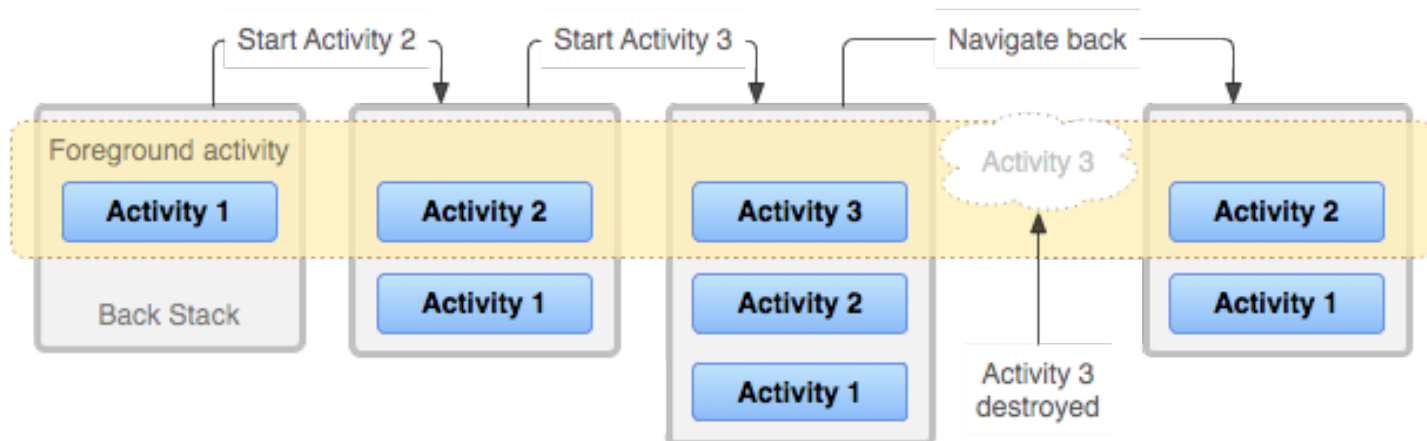




- Serwisy – Service
(działają w tle i nie mają GUI)
- Broadcast Receiver – nasłuchują zdarzeń
(takich jak słaba bateria czy restart urządzenia)
- Content Provider – udostępniają dane
(warstwa pośrednicząca)

Procesy i zadania

- Domyślnie, komponenty wchodzące w skład pojedynczego programu (.apk) uruchamiane są w tym samym **procesie**
- **Zadanie** (ang. task) jest to logicznie powiązana praca, zawierająca własny stos aktywności
- Pojedyncza aktywność może być tworzona wiele razy i używana w ramach różnych zadań (ale istnieje możliwość wymuszenia wielokrotnego wykorzystania tej samej instancji za pomocą pola *android:launchMode* w pliku *AndroidManifest.xml*)





- Priorytet
 1. Proces foreground (np. aktywna aktywność)
 2. Proces widoczny (widoczna aktywność)
 3. Proces z serwisem
 4. Proces z niewidoczną aktywnością (aktywność w stanie stopped)
 5. Proces pusty (brak komponentów – mogą istnieć z uwagi na optymalizację czasu uruchamiania)
- Wyższy priorytet procesu zmniejsza szansę zabicia go przez system



- Aplikacje domyślnie nie mają uprawnień do niektórych działań, takich jak połączenie z Internetem czy dostęp do listy kontaktów
- Istnieją dwa rodzaje uprawnień:
 - Nadawane w trakcie instalacji aplikacji (przed Androidem 6 był to jedyny sposób)
 - Nadawane podczas działania aplikacji (możliwość działania aplikacji w ograniczonym zakresie)
- Wszystkie uprawnienia muszą być zadeklarowane w AndroidManifest.xml (przed węzłem *<application>*):

```
<uses-permission  
    android:name="android.permission.RECORD_AUDIO" />
```



- Pomimo odpowiedniej deklaracji w XML, wiele uprawnień może być nadanych dopiero w trakcie działania aplikacji:

```
if (ContextCompat.checkSelfPermission(this,
    android.Manifest.permission.RECORD_AUDIO) ==
    PackageManager.PERMISSION_DENIED) {
    ActivityCompat.requestPermissions(this,
        arrayOf(android.Manifest.permission.RECORD_AUDIO), 0)
}

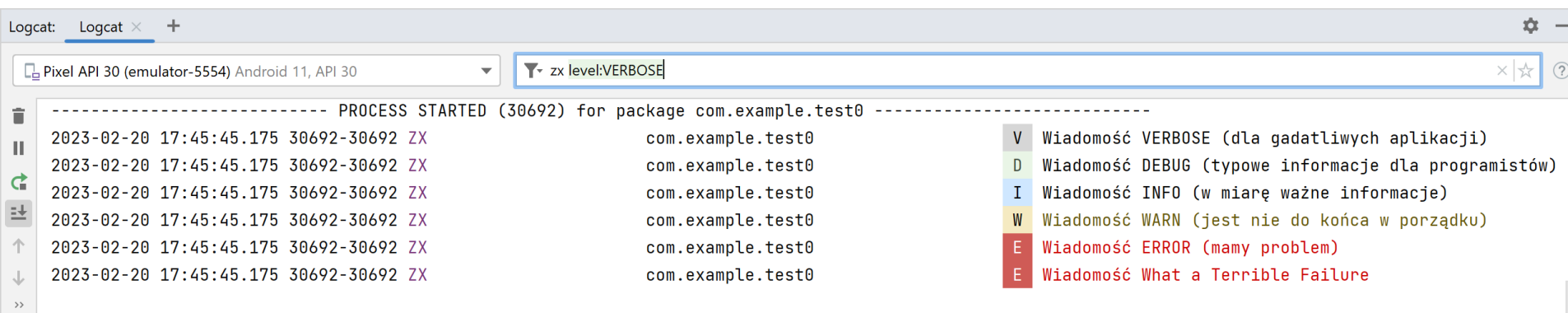
override fun onRequestPermissionsResult(requestCode: Int,
    permissions: Array<String>, grantResults: IntArray) {
    super.onRequestPermissionsResult(requestCode, permissions,
        grantResults)
    if (requestCode == 0) {
        // TU NALEŻY SPRAWDZIĆ, CZY UPRAWNIENIE ZOSTAŁO NADANE
    }
}
```

- Nie jest to wymagane np. dla połączenia z Internetem
- Użytkownik może zmienić uprawnienia w ustawieniach



Różne poziomy logowania

```
Log.v("ZX", "Wiadomość VERBOSE (dla gadatliwych aplikacji)");  
Log.d("ZX", "Wiadomość DEBUG (typowe informacje dla programistów)");  
Log.i("ZX", "Wiadomość INFO (w miarę ważne informacje)");  
Log.w("ZX", "Wiadomość WARN (jest nie do końca w porządku)");  
Log.e("ZX", "Wiadomość ERROR (mamy problem)");  
Log.wtf("ZX", "Wiadomość What a Terrible Failure");
```





Mobilne systemy operacyjne: Interfejs graficzny w systemie Android

Marek Kulawiak, Krzysztof Bruniecki

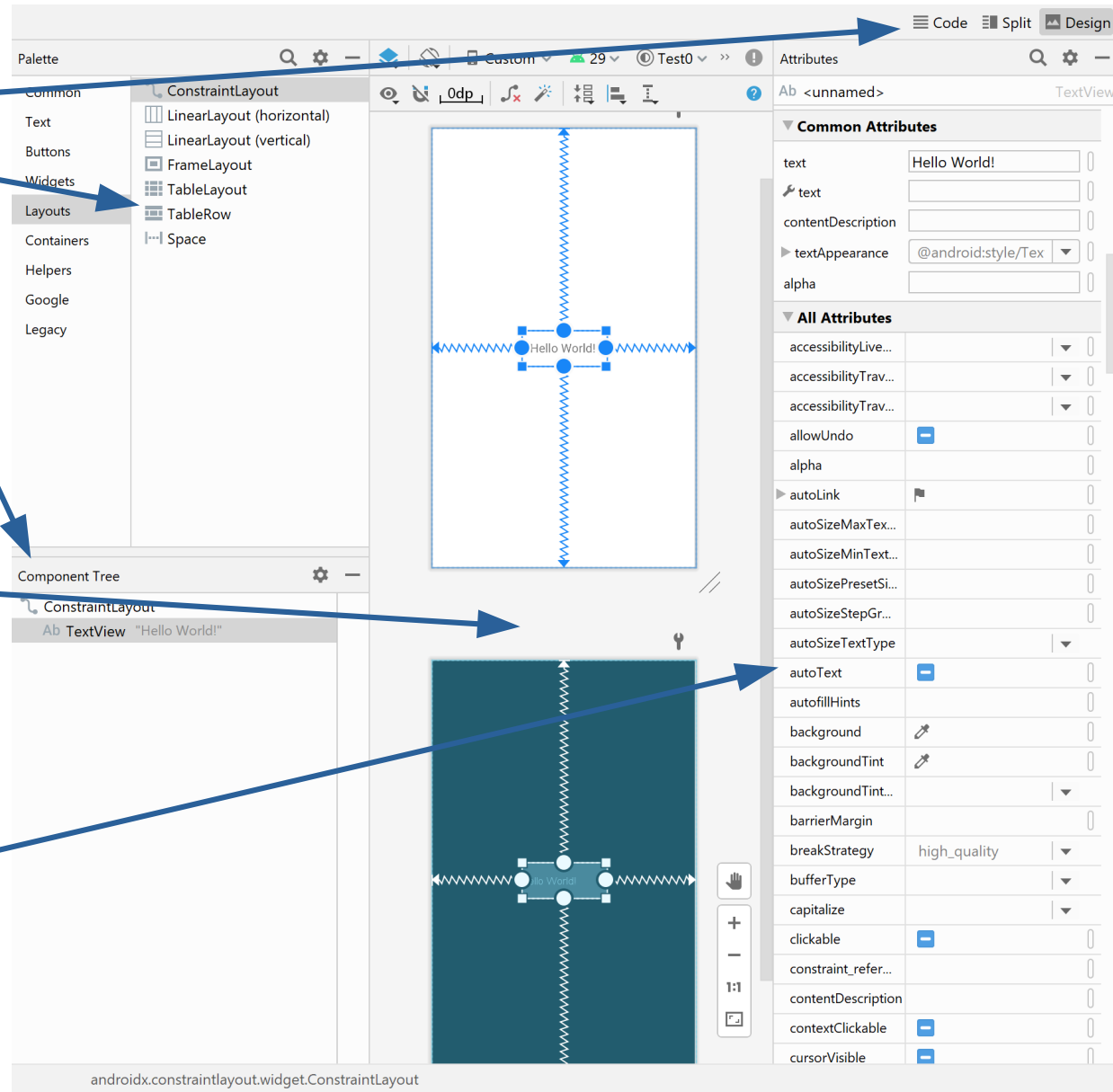


- GUI definiowany w plikach XML lub w kodzie Java/Kotlin
- XML zapewnia większą separację od logiki
- Java i Kotlin dają możliwość tworzenia instrukcji warunkowych i pętli
- Najlepiej łączyć obie techniki (np. zdefiniować kontrolkę w XML i ją programistycznie powielić)



Edycja GUI w Android Studio

- Rodzaj widoku
- Wybór kontrolki
- Hierarchia komponentów
- Edytor w dwóch wariantach (Design oraz Blueprint)
- Atrybuty zaznaczonego komponentu





Zawartość prostego układu w XML

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:padding="24sp"
        android:text="@string/app_name"
        android:textSize="30sp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```



Zawartość prostego okna Compose

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ComposeTestTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        text = stringResource(R.string.app_name),
                        modifier = Modifier.fillMaxSize().padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(text: String, modifier: Modifier = Modifier) {
    Column(
        modifier = modifier,
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = text,
            modifier = Modifier.padding(32.dp)
        )
    }
}
```



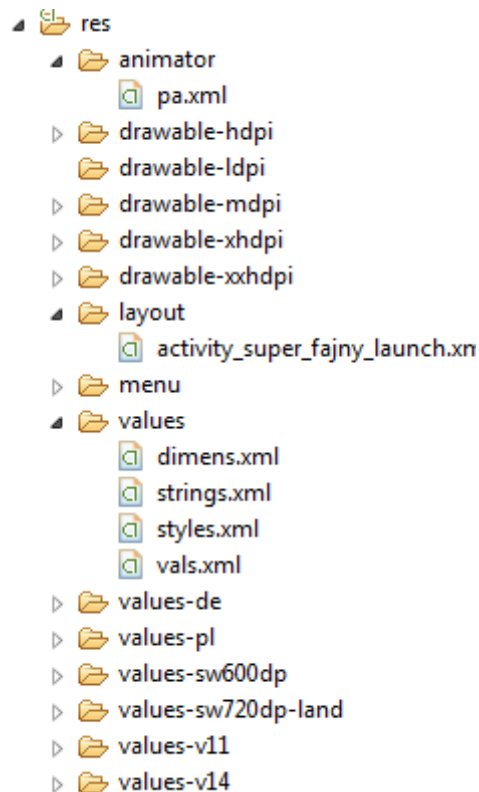
- Zasoby definiowane w oddzielnych plikach
 - Pliki XML z wartościami
 - Łańcuchy znaków
 - Kolory
 - Wartości liczbowe
 - Wymiary elementów GUI

```
<!-- res/values/strings.xml -->  
<resources>  
    <string name="app_name">Nazwa aplikacji</string>  
</resources>
```



Lokalizacja zasobów

- Dla każdego obsługiwanego języka należy dodać osobny katalog *values*-z własnym plikiem *strings.xml*
- Gdy brakuje tekstu dla preferowanego języka, wówczas użyty będzie jego odpowiednik z domyślnego katalogu *values*
- Można również nadpisać inne zasoby aplikacji, np. układy graficzne dostosowane do konkretnych API czy rozdzielczości



```
<resources>
<string-array translatable="false" name="supported_locales">
    ...
    <item>pa-Arab-PK</item> <!-- Punjabi (Arabic, Pakistan) -->
    <item>pa-Guru-IN</item> <!-- Punjabi (Gurmukhi, India) -->
    <item>pl-PL</item> <!-- Polish (Poland) -->
    <item>ps-AF</item> <!-- Pashto (Afghanistan) -->
    <item>pt-AO</item> <!-- Portuguese (Angola) -->
    ...
</string-array>
</resources>
```

Fragment pliku *frameworks/base/core/res/res/values/locale_config.xml*,
pobranego ze źródeł systemu Android (cs.android.com)



Zasoby Compose i tradycyjne

Android ▾

▼ app

▼ manifests

AndroidManifest.xml

▼ kotlin+java

▼ ksg.mso.layoutandcompose

▼ ui.theme

Color.kt

Theme.kt

Type.kt

MainActivity

MainActivity2.kt

> ksg.mso.layoutandcompose (androidTest)

> ksg.mso.layoutandcompose (test)

▼ res

> drawable

▼ layout

activity_main.xml

> mipmap

▼ values

colors.xml

strings.xml

▼ themes (2)

themes.xml

themes.xml (night)

> xml

Color.kt ×

```
1 package ksg.mso.layoutandcompose.ui.theme
2
3 import androidx.compose.ui.graphics.Color
4
5 val Purple80 = Color( color: 0xFFD0BCFF)
6 val PurpleGrey80 = Color( color: 0xFFCCC2DC)
7 val Pink80 = Color( color: 0xFFEB8C80)
8
9 val Purple40 = Color( color: 0xFF6650a4)
10 val PurpleGrey40 = Color( color: 0xFF625b71)
11 val Pink40 = Color( color: 0xFF7D5260)
```

</> colors.xml ×

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <resources>
3   <color name="black">#FF000000</color>
4   <color name="white">#FFFFFFFF</color>
5 </resources>
```




Obsługa przycisku (wariant I)

```
<!-- XML -->
```

```
<Button
```

```
    android:id="@+id/myButton"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_marginTop="8dp"
```

```
    android:text="Button"
```

```
    app:layout_constraintEnd_toEndOf="parent"
```

```
    app:layout_constraintStart_toStartOf="parent"
```

```
    app:layout_constraintTop_toTopOf="parent" />
```

```
// Kotlin
```

```
val button = findViewById<Button>(R.id.myButton)
```

```
button.setOnClickListener {
```

```
    Log.d("ZX", "button.setOnClickListener")
```

```
}
```



```
<!-- XML -->
```

```
<Button
```

```
    android:id="@+id/myButton"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:layout_marginTop="8dp"
```

```
    android:text="Button"
```

```
    app:layout_constraintEnd_toEndOf="parent"
```

```
    app:layout_constraintStart_toStartOf="parent"
```

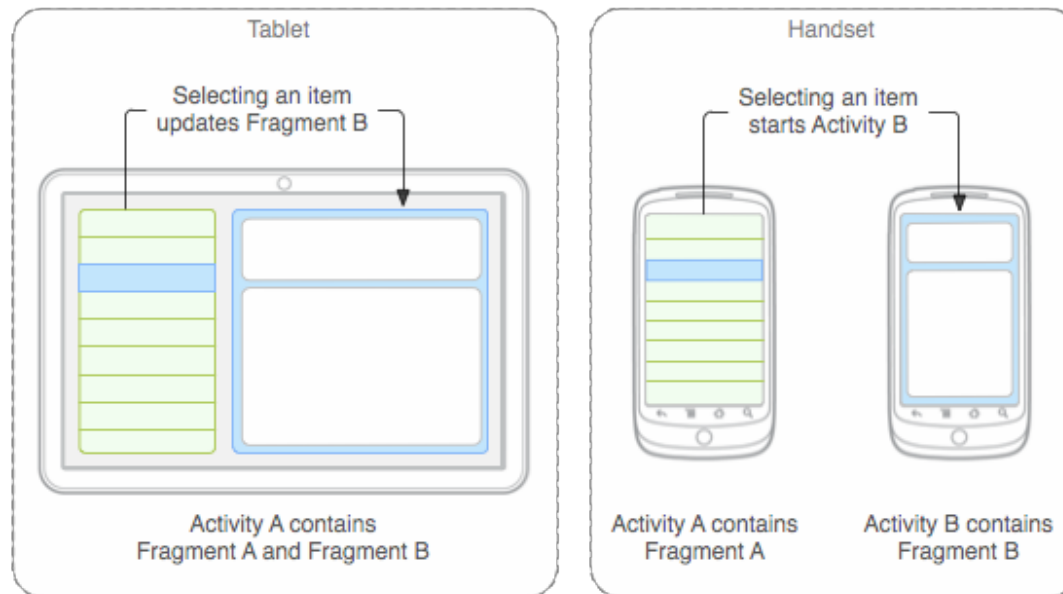
```
    app:layout_constraintTop_toTopOf="parent"
```

```
    android:onClick="onClick" />
```

```
// Kotlin
```

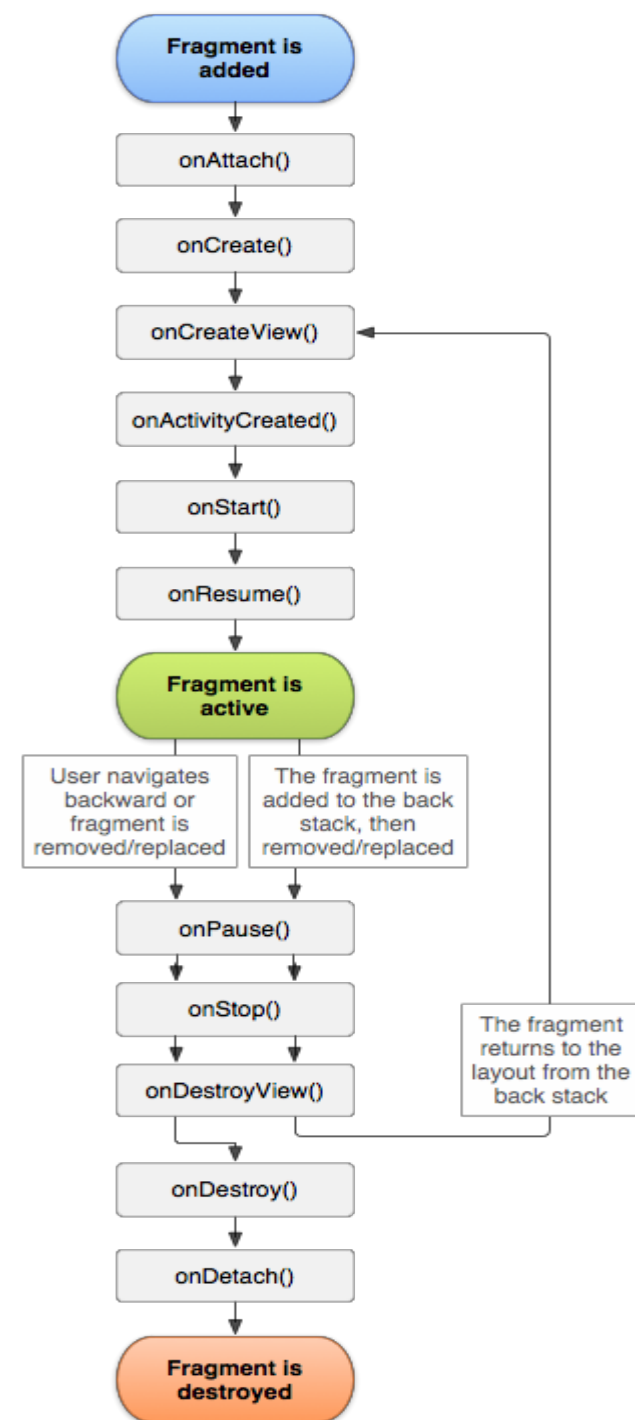
```
fun onClick(view: View) {  
    Log.d("ZX", "onClick")  
}
```

- Fragment reprezentuje porcję funkcji dostarczaną przez aktywność
- Może być użyty w wielu aktywnościach





- Fragmenty charakteryzują się cyklem życia podobnym do cyklu życia aktywności
- Fragmenty można osadzać statycznie w XML lub dynamicznie





- Normalnie gdy aktywność jest odtwarzana fragmenty również są odtwarzane, ale istnieje sposób aby fragment zachowywał swój stan i nie był niszczone
 - `Fragment.setRetainInstance(true)`
 - Powoduje że system nie niszczy fragmentu gdy aktywność jest odtwarzana
 - Metody `OnCreate` i `onDestroy` dla fragmentu nie są wołane



Przykład: fragment do wyświetlania listy

ID1	Content1
ID2	Content2
ID3	Content3
AID1	Added1

```
class ItemFragment : Fragment() {
    val myList:ArrayList<SimpleItem> = arrayListOf(
        SimpleItem("ID1", "Content1"),
        SimpleItem("ID2", "Content2"),
        SimpleItem("ID3", "Content3"))

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
    override fun onCreateView(inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?): View? {
        val view = inflater.inflate(
            R.layout.fragment_item_list, container, false)
        if (view is RecyclerView) {
            with(view) {
                layoutManager = LinearLayoutManager(context)
                adapter = MyItemRecyclerViewAdapter(myList)
            }
        }
        return view
    }
}
```



Przykład: przesłanie danych do fragmentu

```
// MainActivity.kt
val myFragmentManager = supportFragmentManager
val myFragmentTransaction =
    myFragmentManager.beginTransaction()
val mFragment = ItemFragment()

val myBundle = Bundle()
val extraItems = arrayListOf(
    ItemFragment.SimpleItem("AID1", "Added1"),
    ItemFragment.SimpleItem("AID2", "Added2")
)
myBundle.putParcelableArrayList("ExtraItems", extraItems)
mFragment.arguments = myBundle
myFragmentTransaction.add(
    R.id.fragmentContainerView, mFragment).commit()

// ItemFragment.kt
arguments?.let {
    val extraItems =
        it.getParcelableArrayList<SimpleItem>("ExtraItems")
    extraItems?.let {
        myList.addAll(extraItems)
    }
}
```

ID1 Content1

ID2 Content2

ID3 Content3

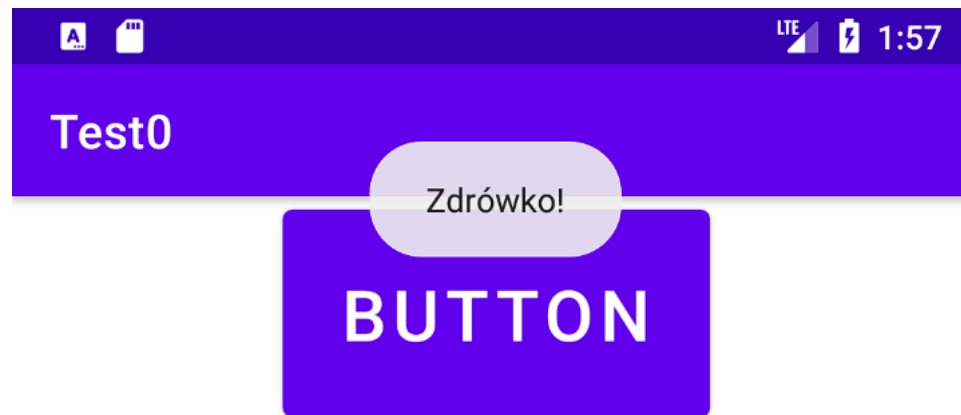
AID1 Added1

AID2 Added2



Toast – proste powiadomienia

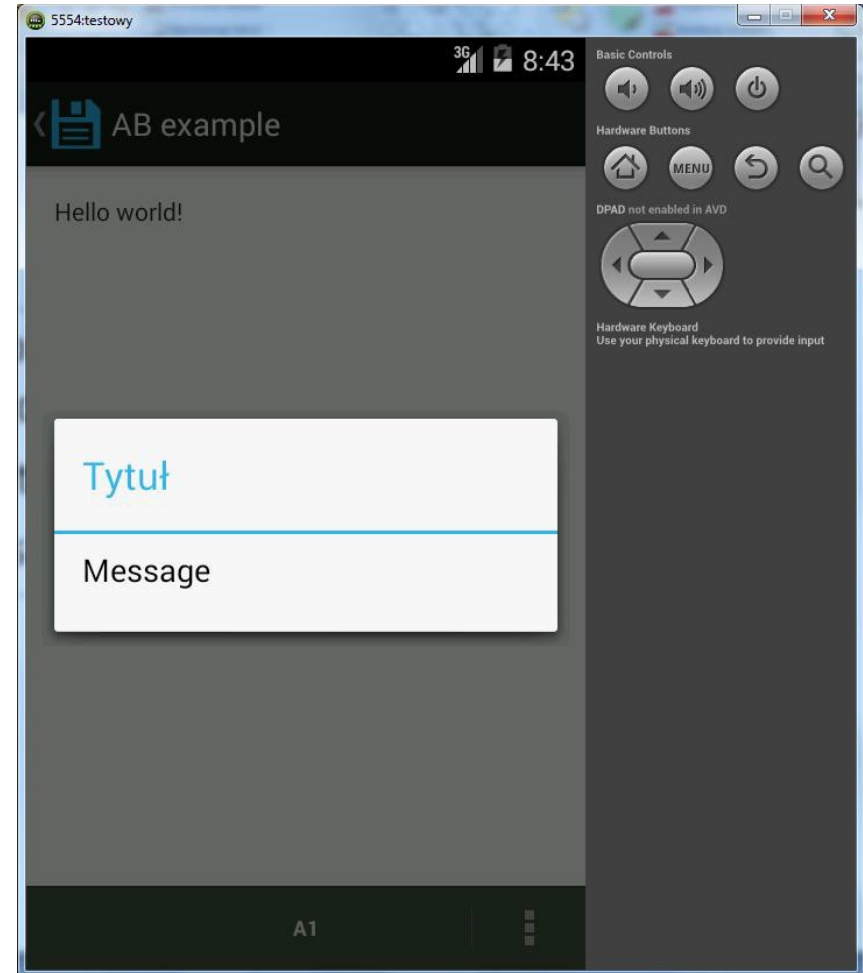
```
val toast = Toast.makeText(this, "Zdrówko!", Toast.LENGTH_LONG)  
toast.setGravity(Gravity.TOP, 0, 0)  
toast.setMargin(0f, 0.05f)  
toast.show()
```





AlertDialog

```
val builder =  
AlertDialog.Builder(this)  
  
builder  
    .setCancelable(true)  
    .setTitle("Tytuł")  
    .setMessage("Message")  
    .create().show()
```

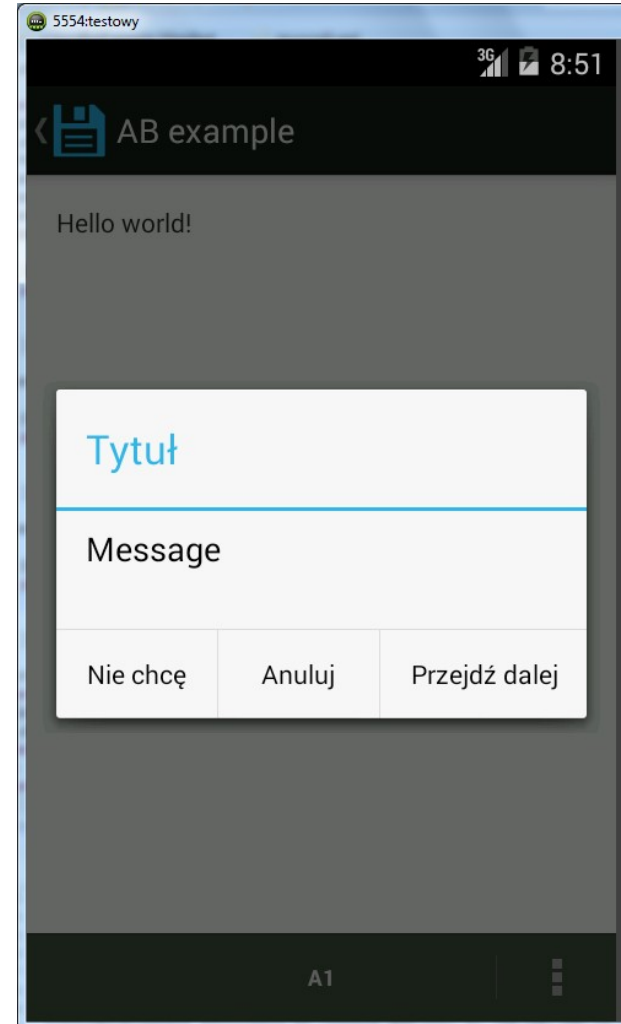




AlertDialog – przyciski

```
val builder = AlertDialog.Builder(this)

builder
    .setPositiveButton("Przejdź dalej")
    { dialog, id ->
        // TODO
    }
    .setNegativeButton("Nie chcę")
    { dialog, id ->
        // TODO
    }
    .setNeutralButton("Anuluj")
    { dialog, id ->
        // TODO
    }
    .create().show()
```

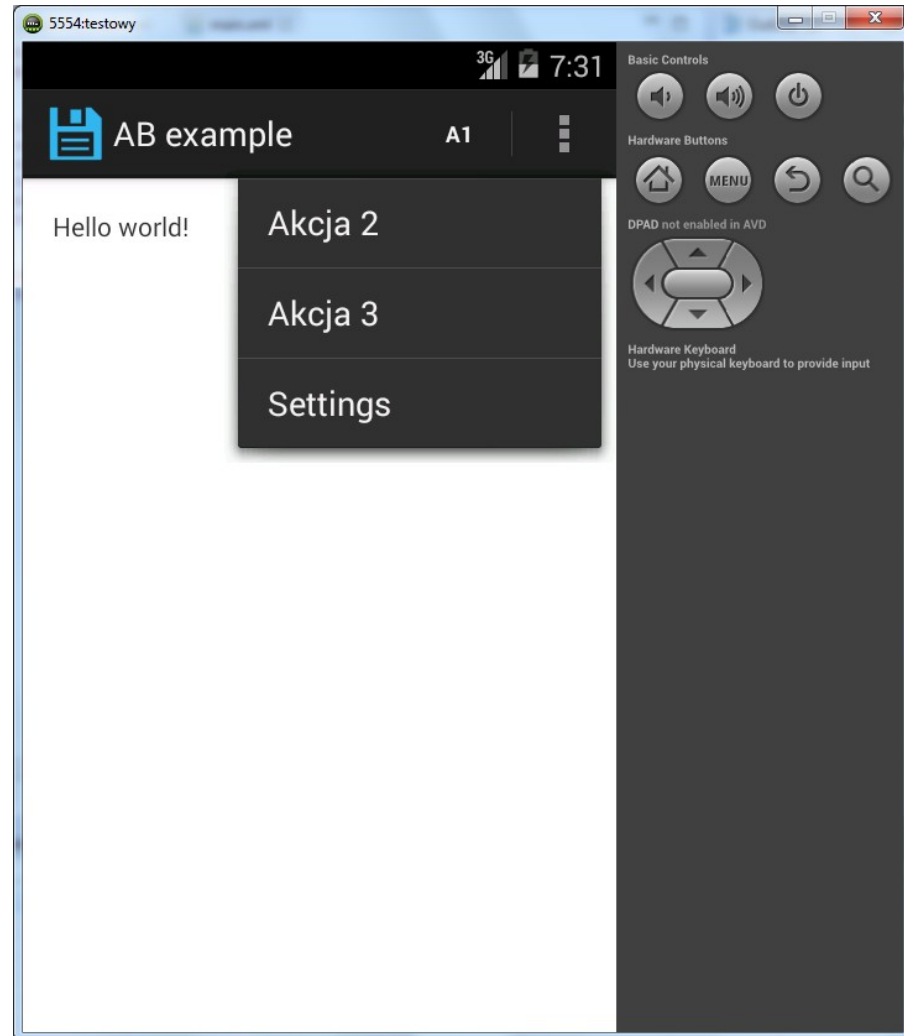




Action Bar – menu wyboru

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main, menu);
    return true;
}
```

```
<menu xmlns:android="
http://schemas.android.com/apk/res/android"
xmlns:yourapp="
http://schemas.android.com/apk/res-auto">
<item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:showAsAction="never"
    android:title="@string/action_settings"/>
<item android:id="@+id/action_a2"
    android:title="Akcja 2"></item>
<item android:id="@+id/action_a3"
    android:title="Akcja 3"></item>
<item android:id="@+id/action_a1"
    android:title="A1"
    android:showAsAction="ifRoom"></item>
</menu>
```





```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    switch(item.getItemId()) {
        case R.id.action_settings:
            Log.d("action_bar", "R.id.action_settings");
            return true;
        case R.id.action_a1:
            Log.d("action_bar", "R.id.action_a1");
            return true;
        case R.id.action_a2:
            Log.d("action_bar", "R.id.action_a2");
            return true;
        case R.id.action_a3:
            Log.d("action_bar", "R.id.action_a3");
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```



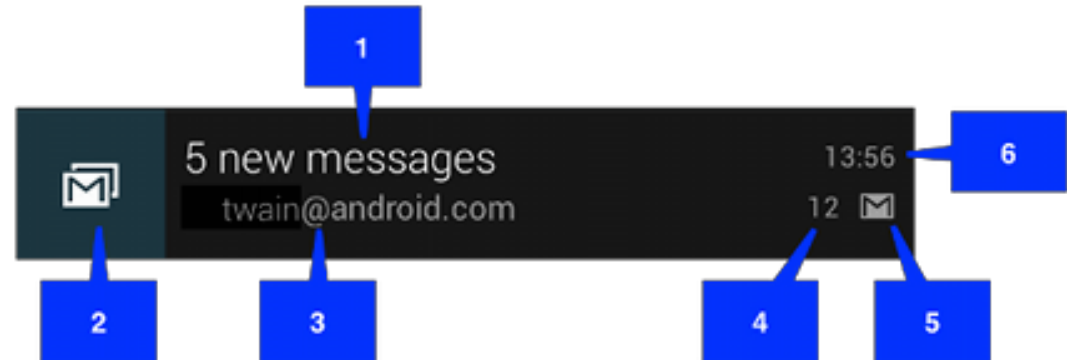
Notifications

Small View:

1. Tytuł
2. Ikona
3. Content text
4. Content info
5. Mała ikona
6. Czas

Big View:

7. Detale





Notifications - przykład

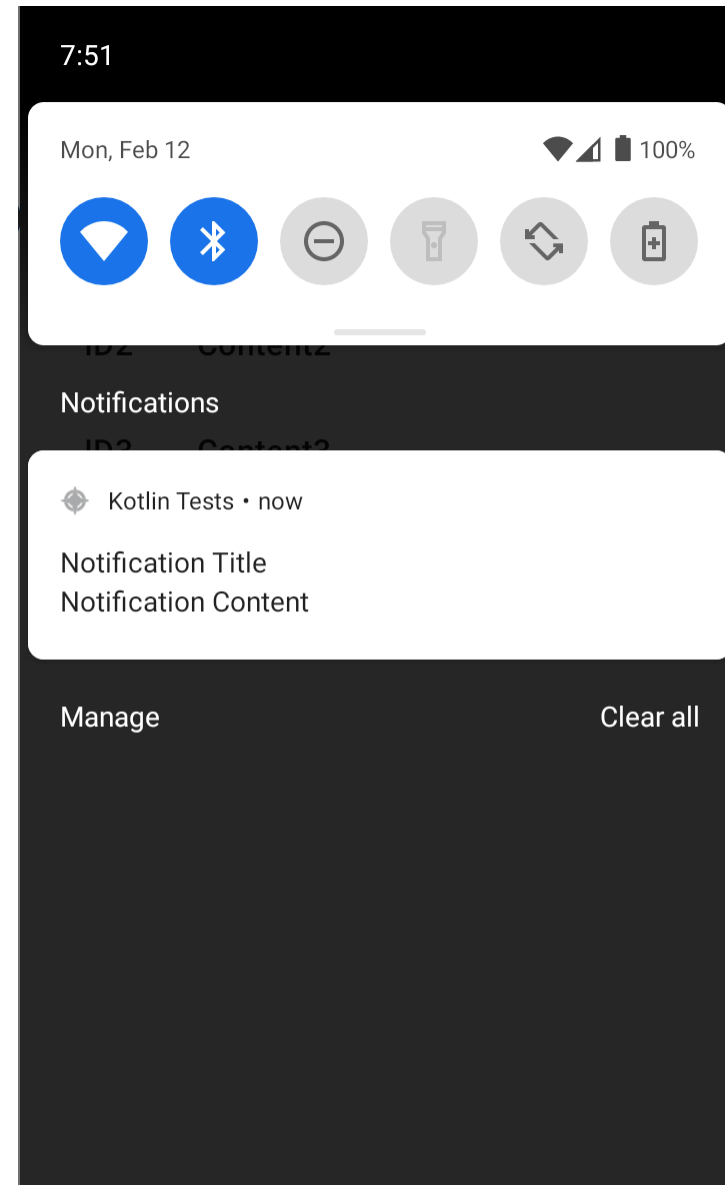
```
val CHANNEL_ID = "channelID"
val NOTIF_ID = 0

val channel = NotificationChannel(
    CHANNEL_ID, "My Channel",
    NotificationManager.IMPORTANCE_DEFAULT)

val notifManager
    = getSystemService(NOTIFICATION_SERVICE)
    as NotificationManager
notifManager.createNotificationChannel(channel)

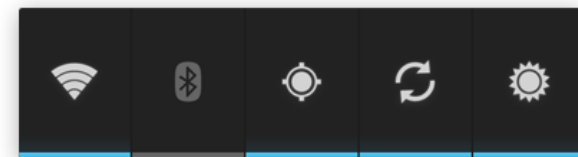
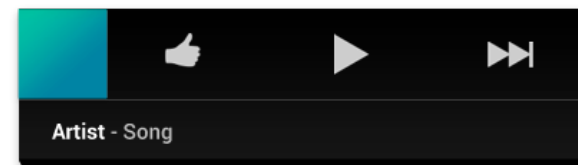
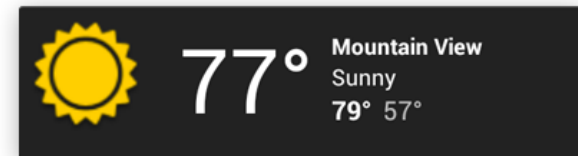
val notif = NotificationCompat
    .Builder(this, CHANNEL_ID)
        .setContentTitle("Notification Title")
        .setContentText("Notification Content")
        .setSmallIcon(android.R.drawable
            .ic_menu_compass)
        .setPriority(NotificationCompat
            .PRIORITY_HIGH)
        .build()

NotificationManagerCompat.from(this)
    .notify(NOTIF_ID, notif)
```





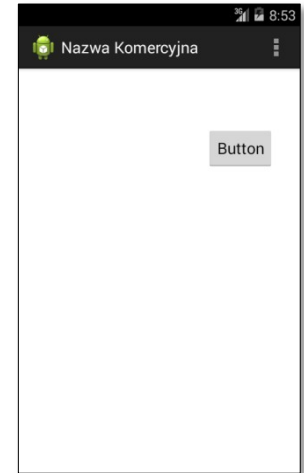
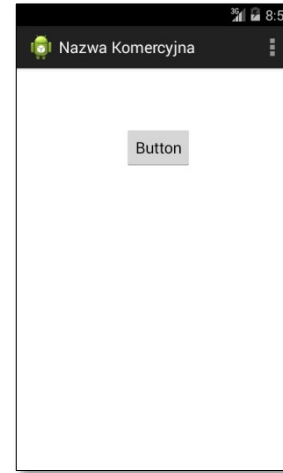
- Umieszczane na pulpicie (Home screen)
- Mogą np. dostarczać informacje albo ułatwiać wykonywanie różnych czynności
- Powinny zezwalać na zmianę rozmiaru (ale nie muszą)
- Mają ograniczoną obsługę gestów (tylko Touch i Vertical swipe)





Property animation

- Możliwość animowania różnych obiektów
 - Kontrolerek
 - Innych kształtów
 - Innych obiektów (nawet niewidocznych)
- Kroki
 - Utworzenie opisu animacji jako zasobu
 - Pobranie animacji i przypisanie do obiektu
 - Wystartowanie animacji



```
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">
  <objectAnimator
    android:propertyName="x"
    android:duration="3000"
    android:repeatCount="infinite"
    android:repeatMode="reverse"
    android:valueTo="400"
    android:valueType="floatType" />
</set>
```

```
Animator an = AnimatorInflater.loadAnimator(this, R.animator.pa);
Button bt = (Button)findViewById(R.id.button1); an.setTarget(bt);
an.start();
```




```
class CustomView : View {  
    constructor(context: Context) : super(context)  
    constructor(context: Context, attrs: AttributeSet) : super(context, attrs)  
    constructor(context: Context, attrs: AttributeSet, defStyleAttr: Int) :  
        super(context, attrs, defStyleAttr)  
  
    override fun draw(canvas: Canvas) {  
        super.draw(canvas)  
        val paint = Paint()  
        paint.setARGB(255, 0, 0, 0)  
        canvas.drawLine(0f, canvas.height.toFloat(),  
                        canvas.width.toFloat(), 0f, paint);  
    }  
}
```

```
<view  
    android:id="@+id/customView"  
    class="ksg.mso.intents.CustomView"  
    android:layout_width="0dp"  
    android:layout_height="0dp"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent" />
```

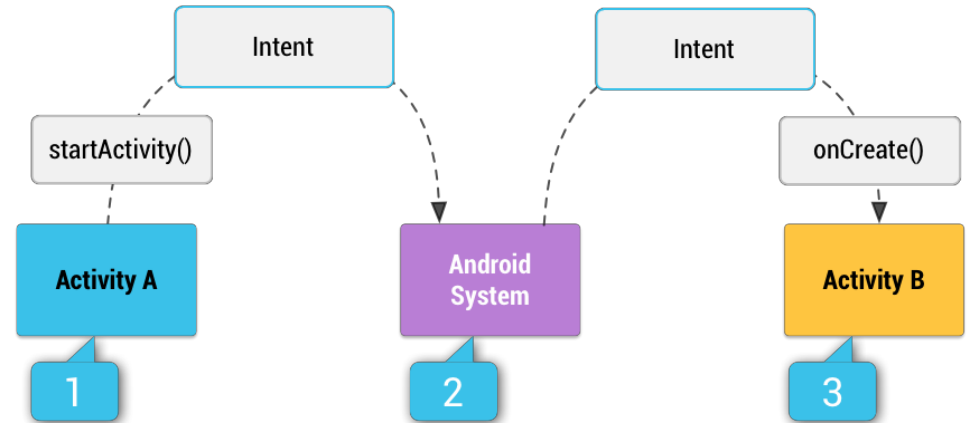


**Mobilne systemy operacyjne:
Komunikacja pomiędzy
aplikacjami w systemie Android**

Krzysztof Bruniecki, Marek Kulawiak



- Struktura danych zawierająca opis akcji do wykonania
- Służy do rozsyłania rozgłoszeń systemowych oraz aktywowania komponentów
- Implementuje interfejsy Parcelable (odpowiednik Serializable) oraz Cloneable





- Intencje dzielą się na dwa rodzaje:
 - “Explicit” - zawierają nazwę komponentu lub pakietu programu, który ma być uruchomiony
 - “Implicit” - opisują rodzaj akcji, która zostanie obsłużona przez kompatybilną aplikację w systemie, wybraną przez użytkownika (aplikacje są dobierane za pośrednictwem “filtrów intencji”)



Przykład Explicit Intent

```
/* MainActivity */
```

```
// Stworzenie intencji:
```

```
val secondActivityIntent =  
    Intent(this, SecondActivity::class.java)
```

```
// Przekazanie opcjonalnych danych w ramach intencji:
```

```
secondActivityIntent.putExtra("JakaśLiczba", 616)
```

```
// Uruchomienie nowej aktywności:
```

```
startActivity(secondActivityIntent)
```

```
/* SecondActivity */
```

```
// Odebranie przekazanych danych:
```

```
val receivedData = intent.getIntExtra("JakaśLiczba", -1)
```



Przykład Implicit Intent

```
// Stworzenie intencji:
val sendIntent = Intent()

// Przygotowanie danych do przesłania:
sendIntent.setAction(Intent.ACTION_SEND)
sendIntent.putExtra(Intent.EXTRA_TEXT, "Zawartość tekstowa.")
sendIntent.setType("text/plain")

// Próba uruchomienia aktywności:
try {
    startActivity(sendIntent)
} catch (e: ActivityNotFoundException) {
    // Gdy w systemie nie ma odpowiedniej aplikacji.
}
```



- Filtry intencji opisują, jakiego rodzaju intencje mogą być obsługiwane przez dany komponent
- Są istotne tylko dla intencji typu Implicit
- Definiowane w pliku AndroidManifest.xml jako węzły `<intent-filter>`
- Dopasowanie do filtru jest sprawdzane na podstawie składowych intencji
 - (Action, Category, Data)
- Jeśli intencja nie dopasuje się do danego filtru, to sprawdzane są kolejne filtry



Intent – Action

- Opis akcji do wykonania
- Rodzaj zgłaszanego zdarzenia dla BroadcastReceiver
- Możliwe jest zdefiniowanie własnych rodzajów akcji

Constant	Target component	Action
<code>ACTION_CALL</code>	activity	Initiate a phone call.
<code>ACTION_EDIT</code>	activity	Display data for the user to edit.
<code>ACTION_MAIN</code>	activity	Start up as the initial activity of a task, with no data input and no returned output.
<code>ACTION_SYNC</code>	activity	Synchronize data on a server with data on the mobile device.
<code>ACTION_BATTERY_LOW</code>	broadcast receiver	A warning that the battery is low.
<code>ACTION_HEADSET_PLUG</code>	broadcast receiver	A headset has been plugged into the device, or unplugged from it.
<code>ACTION_SCREEN_ON</code>	broadcast receiver	The screen has been turned on.
<code>ACTION_TIMEZONE_CHANGED</code>	broadcast receiver	The setting for the time zone has changed.



- Opisuje rodzaj danych (URI oraz typ MIME)
- Gdy aplikacja ma obsłużyć obrazek z pamięci urządzenia:

```
<intent-filter>  
    <data android:mimeType="image/*" />  
    ...  
</intent-filter>
```

- Gdy komponent ma obsłużyć plik video dostępny w sieci:

```
<intent-filter>  
    <data android:scheme="http" android:type="video/*" />  
    ...  
</intent-filter>
```



- Dodatkowe informacje o komponencie
- Wszystkie kategorie wyspecyfikowane w intencji muszą być również w filtrze, żeby intencja dotarła

Constant	Meaning
<code>CATEGORY_BROWSABLE</code>	The target activity can be safely invoked by the browser to display data referenced by a link – for example, an image or an e-mail message.
<code>CATEGORY_GADGET</code>	The activity can be embedded inside of another activity that hosts gadgets.
<code>CATEGORY_HOME</code>	The activity displays the home screen, the first screen the user sees when the device is turned on or when the <i>Home</i> button is pressed.
<code>CATEGORY_LAUNCHER</code>	The activity can be the initial activity of a task and is listed in the top-level application launcher.
<code>CATEGORY_PREFERENCE</code>	The target activity is a preference panel.



Przykład – różne scenariusze uruchomienia

```
<!-- Wyświetlenie ikony na liście aplikacji (App Drawer) -->
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>

<!-- Uruchomienie aplikacji z poziomu przeglądarki internetowej -->
<intent-filter>
    <action
        android:name="android.intent.action.VIEW" />
    <category
        android:name="android.intent.category.DEFAULT" />
    <category
        android:name="android.intent.category.BROWSABLE" />
    <data
        android:scheme="http"
        android:host="firma.pl"
        android:path="/aplikacja" />
</intent-filter>
```



```
// JavaScript
```

```
var testString = 'test';  
var href =  
    'intent://firma.pl/aplikacja#Intent;  
    scheme=http;package=pl.firma.aplikacja;  
    S.jakis_string=' + testString + ';end;';  
  
document.getElementById("linkButton").setAttribute('href', href);
```

```
// MainActivity.java
```

```
Bundle parameters = getIntent().getExtras();  
if (parameters != null)  
{  
    String tempString = parameters.getString("jakis_string");  
    if (tempString != null && !tempString.isEmpty())  
    {  
        ...  
    }  
}
```



Intent – Extras i Flags

- Extras: pary klucz-wartość, dostarczane do komponentu obsługującego intencję
 - np. `intent.putExtra("JakaśLiczba", 199999)`
 - zamiast pojedynczych wartości można przesłać cały obiekt Bundle
- Flags: różne wartości specjalne, zdefiniowane w klasie Intent
 - `FLAG_ACTIVITY_CLEAR_TOP` – jeśli aktywność była już uruchomiona w ramach zadania, to przywróć jej instancję, usuwając przy tym niepotrzebne aktywności z góry stosu
 - `FLAG_ACTIVITY_REORDER_TO_FRONT` – przenieś istniejącą instancję aktywności na górę stosu
 - `FLAG_ACTIVITY_SINGLE_TOP` – nie otwieraj tej aktywności, jeśli już jest ona na wierzchu stosu



```
/* MainActivity */
```

```
// W głównym ciele klasy
```

```
val startForResultActivity =  
registerForActivityResult(ActivityResultContracts  
.StartActivityResult()) { result: ActivityResult ->  
    if (result.resultCode == Activity.RESULT_OK) {  
        val data: Intent? = result.data  
        val returnedValue = data?.getIntExtra("JakiśWynik", -1)  
    }  
}
```

```
// W innym miejscu, np. w logice przycisku
```

```
startForResultActivity.launch(secondActivityResultIntent)
```



```
/* SecondActivity */

// Nadpisanie działania systemowego przycisku "wstecz"
onBackPressedDispatcher.addCallback(this, object:
    OnBackPressedCallback(true) {
        override fun handleOnBackPressed() {
            val data = Intent()
            data.putExtra("JakiśWynik", 1610)
            setResult(Activity.RESULT_OK, data)
            finish()
        }
    })
```



- Specjalny rodzaj intencji
- Uruchomiona aplikacja otrzymuje takie same uprawnienia, jak ta która ją wywołała
- Przykłady zastosowania:
 - utworzenie notyfikacji ze specjalnymi uprawnieniami
 - nadanie specjalnych uprawnień App Widgetowi



**Mobilne systemy operacyjne:
System plików w systemie
Android**

Krzysztof Bruniecki, Marek Kulawiak



System plików w systemie Android

- Hierarchiczny system plików typowy dla UNIX/Linux
- Różne typy plików
 - Pliki zwykłe (dane) i katalogi
 - Pliki specjalne (wejścia/wyjścia urządzeń)
 - Gniazda i nazwane potoki (IPC)
 - Dowiązania symboliczne
- Podział na partycje (mount)
 - W pierwszych wersjach użycie YAFFS (Yet Another Flash File System), przystosowany do specyfiki pamięci NAND
 - Od 2010 Adaptacja standardowych systemów plików (tj. ext4)





- Dostęp z poziomu urządzenia:
 - Pliki w pamięci zarezerwowanej dla aplikacji
 - Pliki w pamięci ogólnodostępnej
(mogą wymagać specjalnych uprawnień)
 - Lokalna relacyjna baza danych (SQLite)
 - Content Provider
 - Serwery sieciowe (np.: cloud storage)
- Dostęp z zewnętrznego urządzenia (komputera):
 - Android Debug Bridge (adb)
 - Android Studio



(pliki statyczne, bez możliwości modyfikacji)

- Katalog **res/raw**

- dla każdego pliku generowany jest jego identyfikator w klasie R
- `getResources().openRawResource(R.raw.nazwa)`

- Katalog **assets**

- swobodne nazewnictwo plików
- możliwość tworzenia podkatalogów
- `getAssets().list("jakas/sciezka")`



- **Pamięć wewnętrzna**
 - pliki prywatne różnych aplikacji
- **Pamięć zewnętrzna**
 - pliki ogólnodostępne
- **Pamięć zewnętrzna wymienna**
 - pliki zapisane na zewnętrznej karcie SD



- **Pamięć wewnętrzna**

- pliki prywatne różnych aplikacji
 - ◊ (zapisane na wewnętrznej karcie urządzenia)

- **Pamięć zewnętrzna**

- pliki ogólnodostępne
 - ◊ (również zapisane na wewnętrznej karcie)

- **Pamięć zewnętrzna wymienna**

- pliki zapisane na zewnętrznej karcie SD
 - ◊ (obecnie emulowanej na karcie wewnętrznej)



- Zapisywane w specjalnym położeniu, np.:
 - /data/data/<nazwa.pakietu>/files/plik.dat
 - /data/user/0/<nazwa.pakietu>/files/test.txt
- Usuwane przy odinstalowaniu aplikacji
- Przykłady dostępu:
 - Context.getFilesDir()
 - openFileOutput(
 FILENAME, Context.MODE_PRIVATE);
- Pliki preferencji (klucz-wartość):
 - getActivity().getPreferences(Context.MODE_PRIVATE);



- Dostęp do plików na karcie wbudowanej:
 - `Environment.getExternalStorageDirectory()`
 - `System.getenv("EXTERNAL_STORAGE")`
- Przykładowe ścieżki do pliku:
 - `/mnt/sdcard/file.txt`
 - `/sdcard/file.txt`
 - `/storage/emulated/0/file.txt`
 - `/document/primary:file.txt` (URI)



- Dostęp do plików na karcie SD:
 - `Context.getExternalFilesDirs(String type)`
 - ◊ type oznacza podkatalog
 - ◊ np. `Environment.DIRECTORY_PICTURES`
 - ◊ `type == null` oznacza główny katalog
- Ścieżka:
 - `/storage/<identyfikator_karty>/file.txt`
 - `/document/<identyfikator_karty>:file.txt` (URI)



- Dla systemów Android w wersji ≤ 10 :
 - dostęp bezpośredni za pomocą API File (Java)
 - READ_ / WRITE_ EXTERNAL_STORAGE
- Dla Android 11+:
 - READ_EXTERNAL_STORAGE (tylko odczyt)
 - MANAGE_EXTERNAL_STORAGE
 - ◊ część tzw. “Restricted Permissions” w Google Play
 - ◊ przykłady dozwolonego użycia: menadżer plików, antywirus, szyfrowanie, tworzenie kopii zapasowych
- Alternatywa: Storage Access Framework
 - wybór plików przez wbudowanego menadżera
 - identyfikowanie plików po URI
 - mniejsza swoboda, za to nie wymaga uprawnień



- „Lekki” silnik bazy danych
- Większość klasycznego standardu SQL-92
 - Transakcje
 - Wyzwalacze (ang. triggers)
 - Funkcje
 - ...
- Baza przechowywana w pliku
 - np. /data/data/pakiet.com/plik.db



- Na emulatorze (czasem również na urządzeniu) z systemem Android znajduje się narzędzie sqlite3, pozwalające na uzyskanie dostępu do bazy danych z poziomu konsoli

```
adb -e shell
```

```
# sqlite3 /data/data/com.example.firstdatabase/databases/bruno.db
```

```
sqlite> .schema
```

```
.schema
```

```
CREATE TABLE android_metadata (locale TEXT);
```

```
CREATE TABLE featuretypes (_ID INTEGER PRIMARY KEY, FeatureName  
    VARCHAR);
```



```
public static SQLiteDatabase SQLiteDatabase.openDatabase  
    _ (String path,  
      SQLiteDatabase.CursorFactory factory, int flags)
```

- path – ścieżka do pliku
- factory – opcjonalna fabryka, jeśli chcemy zastąpić domyślny kursor klasy SQLiteCursor, lub **null**
- flags – tryb otwierania/tworzenia
 - OPEN_READWRITE
 - OPEN_READONLY
 - CREATE_IF_NECESSARY
 - NO_LOCALIZED_COLLATORS

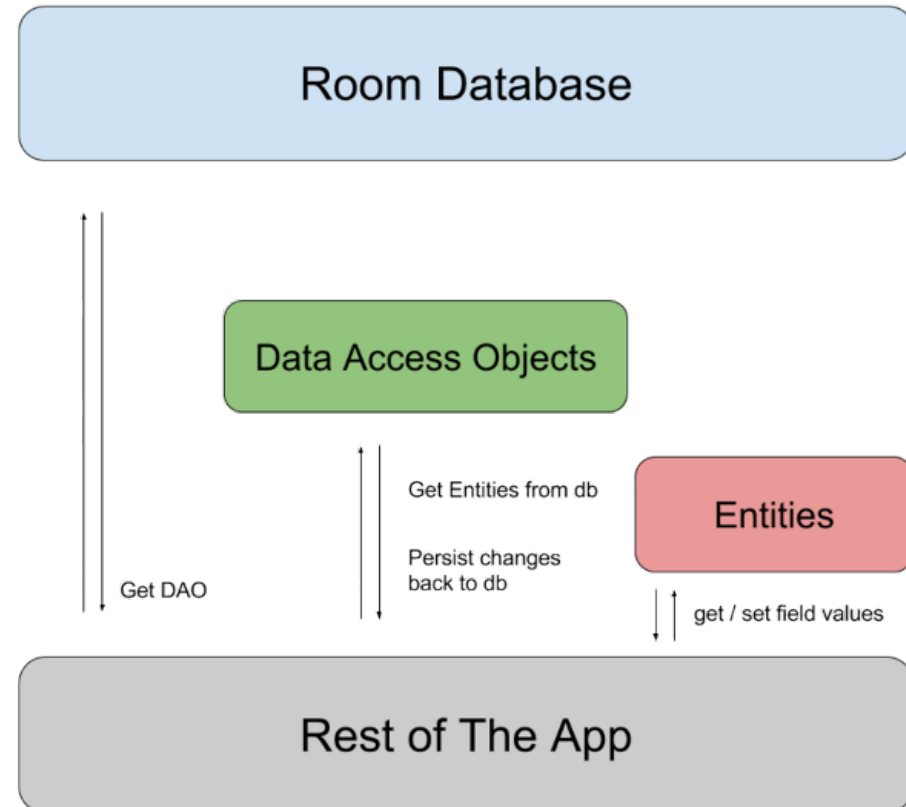


```
class PoiDatabaseHelper extends SQLiteOpenHelper {
    private static final String SQL_CREATE_POI = "CREATE TABLE " +
        "poi (" +
        "    _ID INTEGER PRIMARY KEY, " +
        "    DESC TEXT," +
        "    LON10E7 INTEGER, " +
        "    LAT10E7 INTEGER );";
    public PoiDatabaseHelper(Context context, String name, CursorFactory factory,
        int version) {
        super(context, name, factory, version);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(SQL_CREATE_POI);
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE poi");
        db.execSQL(SQL_CREATE_POI);
    }
}

final String DBNAME = "geobaza";
mOpenHelper = new PoiDatabaseHelper(getContext(), DBNAME, null, 2);
SQLiteDatabase sqlDb = mOpenHelper.getReadableDatabase();
Cursor cur = sqlDb.rawQuery("SELECT * from poi", null); // bez średnika
```

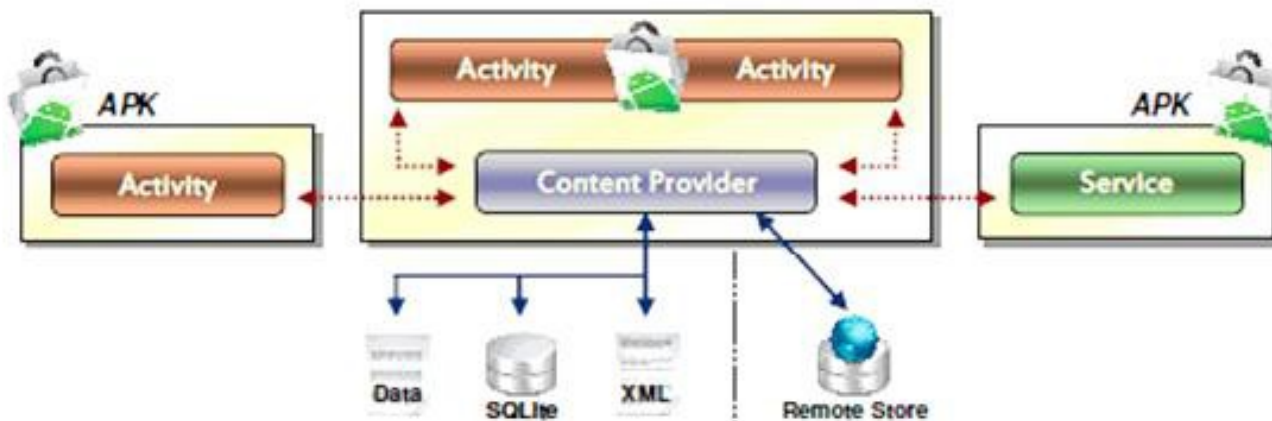


- Nakładka na SQLite
 - weryfikuje zapytania SQL podczas kompilacji
 - stosuje adnotacje w celu zredukowania ilości powtarzalnego kodu





- Pośrednicząca warstwa zarządzająca dostępem do ustrukturyzowanych zasobów danych
 - Udostępnia dane innym aplikacjom
 - ◊ pliki, bazy danych, itp.
 - Dostęp do danych w formie tabel
 - ◊ query, insert, update, delete, getType, onCreate





- Komunikuje się z ContentProvider i udostępnia metody “CRUD” (create, retrieve, update, delete)
- Obiekt pobierany z kontekstu aplikacji metodą `getContentResolver()`
- **`public final Cursor query (Uri uri, String[] projection, String selection, String[] selectionArgs, String sortOrder)`**
 - **`projection`** – lista wybranych kolumn
 - **`selection`** – fraza jak w SQL (bez WHERE) z pytajnikami (?) zastępowanymi **`selectionArgs`**
 - **`sortOrder`** – fraza jak w SQL (bez ORDER BY)



query (CP) a SELECT (SQL)

query() argument	SELECT keyword/parameter	Notes
Uri	FROM <i>table_name</i>	Uri maps to the table in the provider named <i>table_name</i> .
projection	<i>col,col,col,...</i>	projection is an array of columns that should be included for each row retrieved.
selection	WHERE <i>col</i> = <i>value</i>	selection specifies the criteria for selecting rows.
selectionArgs	(No exact equivalent. Selection arguments replace ? placeholders in the selection clause.)	
sortOrder	ORDER BY <i>col,col,...</i>	sortOrder specifies the order in which rows appear in the returned Cursor .



**Mobilne systemy operacyjne:
Android – serwisy, wątki, Interface Definition Language**

Krzysztof Bruniecki, Marek Kulawiak



- Priorytet
 1. Proces foreground (np. aktywna aktywność)
 2. Proces widoczny (widoczna aktywność)
 3. Proces z serwisem
 4. Proces z niewidoczną aktywnością (aktywność w stanie stopped)
 5. Proces pusty (brak komponentów – mogą istnieć z uwagi na optymalizację czasu uruchamiania)
- Wyższy priorytet procesu zmniejsza szansę zabicia go przez system



- Domyślnie cała logika aplikacji odbywa się w pojedynczym wątku
 - (tym samym który obsługuje GUI)
- Żeby nie zablokować działania interfejsu, należy utworzyć odrębny wątek
 - Job, Thread, Runnable, AsyncTask
- Dla długotrwałych operacji warto rozważyć użycie serwisów...



- Serwis jest komponentem działającym w tle
 - będzie działał nawet gdy użytkownik przejdzie do innej aplikacji
 - może być używany do komunikacji z innymi komponentami, także w osobnych aplikacjach
 - z założenia nie posiada interfejsu graficznego, ale może wyświetlać notyfikacje...



- Wyświetla notyfikację w belce statusu
 - np. odgrywanie muzyki, pobieranie pliku
 - możliwość wywołania akcji po kliknięciu w ikonę
- Pozwala na korzystanie z uprawnień niedostępnych dla serwisów typu Background
 - mikrofon, kamera, nawigacja satelitarna
- Ma wyższy priorytet procesu niż zwykły serwis
- Przełączanie trybu działania za pomocą metod **startForeground()** i **stopForeground(boolean removeNotify)**



- FOREGROUND_SERVICE_CAMERA
- FOREGROUND_SERVICE_CONNECTED_DEVICE
- FOREGROUND_SERVICE_DATA_SYNC
- FOREGROUND_SERVICE_HEALTH
- FOREGROUND_SERVICE_LOCATION
- FOREGROUND_SERVICE_MEDIA_PLAYBACK
- FOREGROUND_SERVICE_MICROPHONE
- FOREGROUND_SERVICE_PHONE_CALL
- FOREGROUND_SERVICE_SPECIAL_USE

```
<service  
    android:name=".MyService"  
    android:enabled="true"  
    android:exported="false"  
    android:foregroundServiceType="dataSync"  
    android:stopWithTask="true" />
```




Started

- Po uruchomieniu klient nie ma z nim łączności
- Dobry do wykonania pojedynczej, czasochłonnej operacji
- Uruchamiany przez inny komponent poprzez **startService(Intent)**
- Będzie działać aż do wywołania metody **stopSelf()** lub **stopService()**
 - może działać nawet gdy zostanie „osierocony” przez komponent który go wystartował

Bound

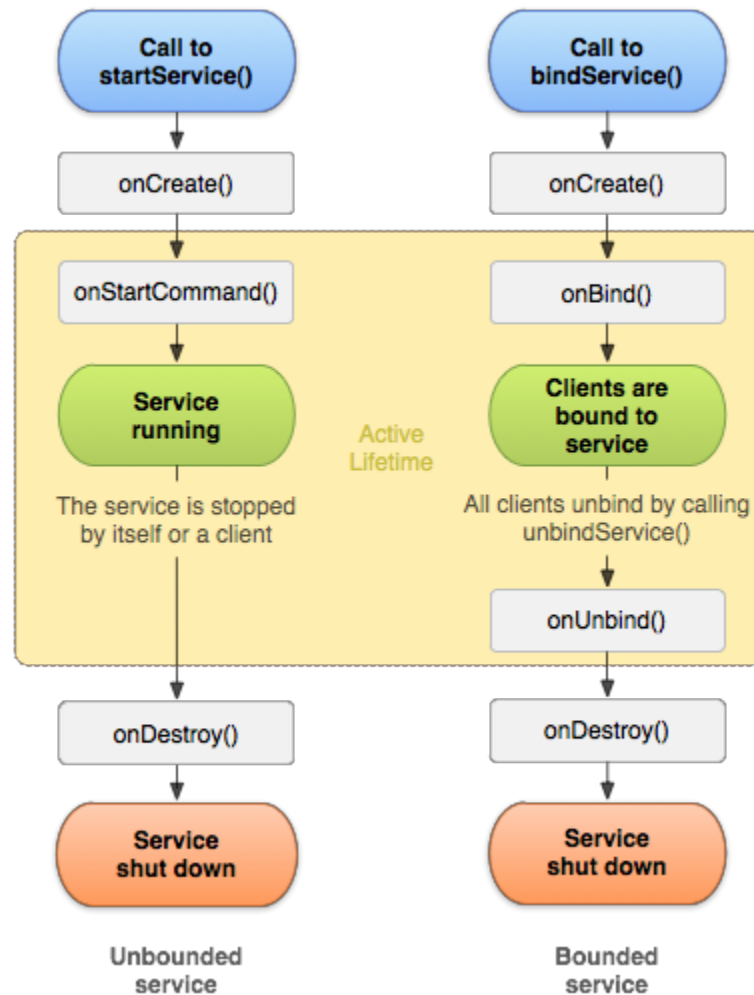
- Dopuszcza komunikację z wieloma klientami naraz (poprzez interfejs IBinder)
- Uruchomiony przy pierwszym wywołaniu metody **bindService(Intent)**
- Działa dopóki nie odłączy się od niego ostatni klient (za pomocą metody **unbindService()**)

Started+Bound

- Działa dopóki podłączeni są klienci ORAZ nie zostanie zastopowany

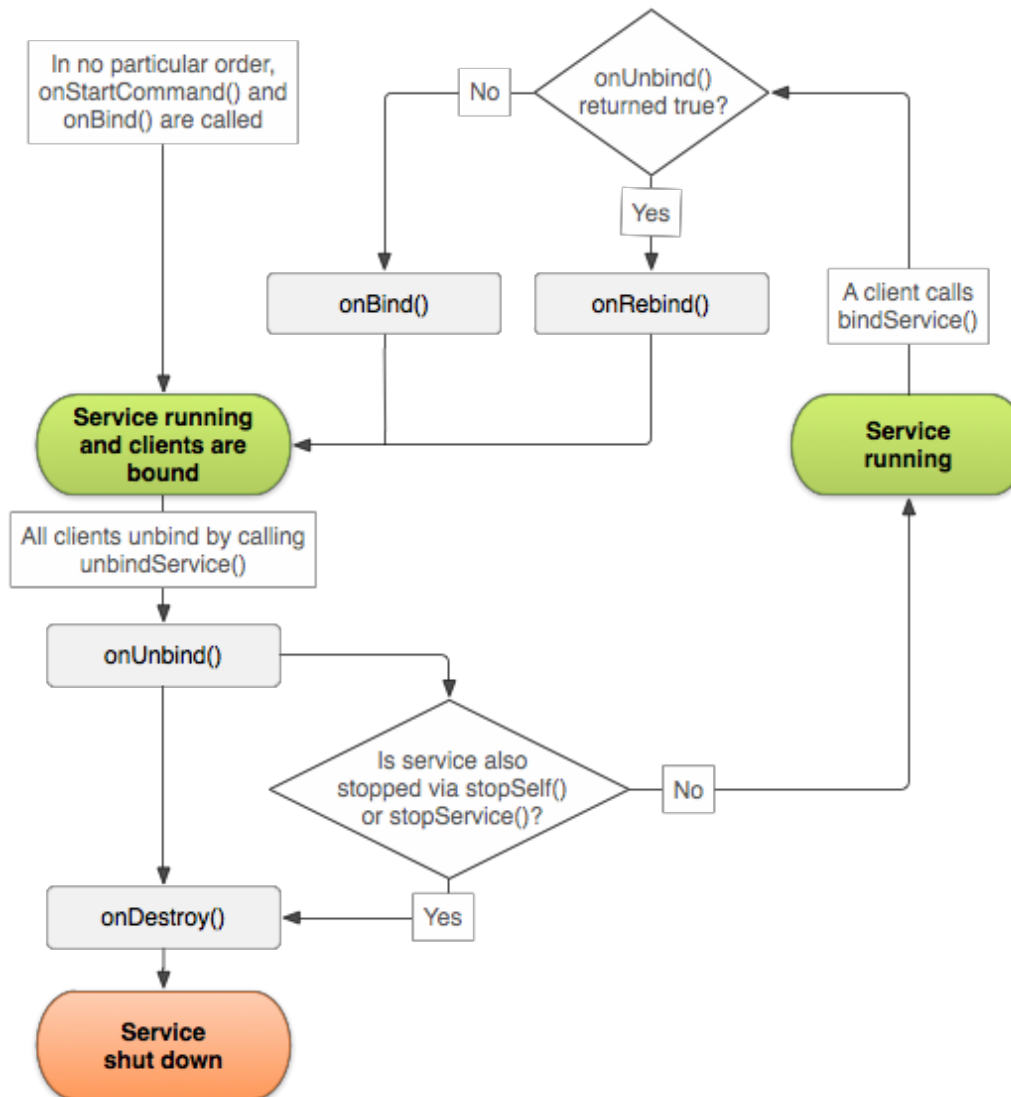


Cykl życia serwisu (Started kontra Bound)





Cykl życia serwisu Bound+Started





- **Service**

- Klasa bazowa dla serwisów
- Nie posiada własnego wątku (należy go utworzyć samodzielnie)
- Przydatna gdy serwis powinien obsługiwać wiele żądań jednocześnie

- **LifecycleService**

- Jak wyżej, ale pozwala na użycie Coroutines

- **IntentService**

- Prostsza w obsłudze klasa dziedzicząca po Service
- Automatycznie tworzy własny wątek oraz zatrzymuje swoje działanie po obsłużeniu żądania
- Właściwa w przypadku, gdy nie ma potrzeby obsługi wielu żądań równocześnie



- Dostęp poprzez wywołanie `getSystemService()`
- Przykłady:
 - `LOCATION_SERVICE` – zwraca obiekt `LocationManager` (do pozyskiwania danych GNSS)
 - `NOTIFICATION_SERVICE` – zwraca `NotificationManager` (do tworzenia i usuwania powiadomień)
 - `POWER_SERVICE` – zwraca `PowerManager` (do “obchodzenia” mechanizmów oszczędzania energii)
 - `DOWNLOAD_SERVICE` – zwraca `DownloadManager` (do długotrwałego pobierania plików)
 - `VIBRATOR_MANAGER_SERVICE` – zwraca `VibratorManager` (do zarządzania wibracjami)



W systemie Android istnieją trzy mechanizmy pośredniczenia w komunikacji pomiędzy serwisami typu Bound:

1. Binder

- Dla wewnętrznych serwisów, prywatnych dla danej aplikacji
- Klient nawiązuje połączenie z serwisem za pomocą metody `bindService()`, w wyniku której serwis zwraca klientowi obiekt klasy dziedziczącej po Binder
- Obiekt Binder z kolei udostępnia referencję serwisu wraz z jego publicznymi metodami, które klient może uruchamiać



2. Messenger

- Gdy potrzebna jest komunikacja pomiędzy różnymi procesami
- Serwis tworzy obiekt klasy **Messenger** w celu odbierania wiadomości po czym zwraca klientowi obiekt **IBinder** pełniący rolę pośrednika
- Klient odbiera **IBinder** i opakowuje go we własny obiekt **Messenger**, przez którego wysyła wiadomości do serwisu
- Jeśli klient chce otrzymać odpowiedź na swoją wiadomość, musi stworzyć osobny obiekt **Messenger** i przekazać go serwisowi
 - (jako parametr *replyTo* wiadomości Message)



3. Android Interface Definition Language (AIDL)

- Jednoczesna obsługa wielu żądań za pomocą ustandaryzowanego interfejsu do komunikacji pomiędzy procesami
- Obiekty są dekomponowane do typów prostych i następuje ich przesłanie między procesami (marshalling)
- Marshalling jest żmudny do implementacji, dlatego jest to wspomagane przez AIDL



- Na potrzeby serwisu konieczne jest przygotowanie pliku .aidl definiującego interfejs (składnia języka Java) z dostępnymi metodami, np.: **SomeAidlInterface.aidl**
- Plik .aidl musi zostać skopiowany do projektów wszystkich aplikacji, które będą chciały się komunikować z powyższym serwisem

```
package ksg.labxaidl;
```

```
interface SomeAidlInterface {  
    int getPid();  
    int calculateSum(int arg1, int arg2);  
}
```



Przykładowy serwis wykorzystujący AIDL

- Należy utworzyć serwis, który w metodzie `onBind()` będzie zwracał instancję klasy implementującej przygotowany interfejs:

```
package ksg.labxaidl;

public class RemoteAidlService extends Service {
    ...
    private final SomeAidlInterface.Stub mBinder
        = new SomeAidlInterface.Stub() {

        public int getPid() {
            return android.os.Process.myPid();
        }

        public int calculateSum(int arg1, int arg2) {
            return arg1 + arg2;
        }

    };
}
```



Domyślne typy w AIDL

- Typy proste:
 - **int**
 - **double**
 - ...
- Tablice
- Niektóre typy złożone
 - **String**
 - **CharSequence**
 - **List**
 - **Map**
- Obiekty przechowywane w liście i mapie muszą być zrozumiałe dla AIDL
 - domyślne
 - inne zdefiniowane w AIDL
 - Parcelable (interfejs do międzyprocesowego przesyłu obiektów)
- Lista i Mapa przekazywane są do roboczego procesu jako konkretne typy **ArrayList** oraz **HashMap**



Implementacja Parcelable

```
public class MyType implements Parcelable {
    double waga;
    int numer;

    public void writeToParcel(Parcel out, int arg1) {
        out.writeDouble(waga);
        out.writeInt(numer);
    }

    public void readFromParcel(Parcel in) {
        waga = in.readDouble(); numer = in.readInt();
    }

    public static Parcelable.Creator<MyType> CREATOR
        = new Parcelable.Creator<MyType>() {
        public MyType createFromParcel(Parcel arg0) {
            MyType mt = new MyType();
            mt.waga = arg0.readDouble();
            mt.numer = arg0.readInt();
            return mt;
        }

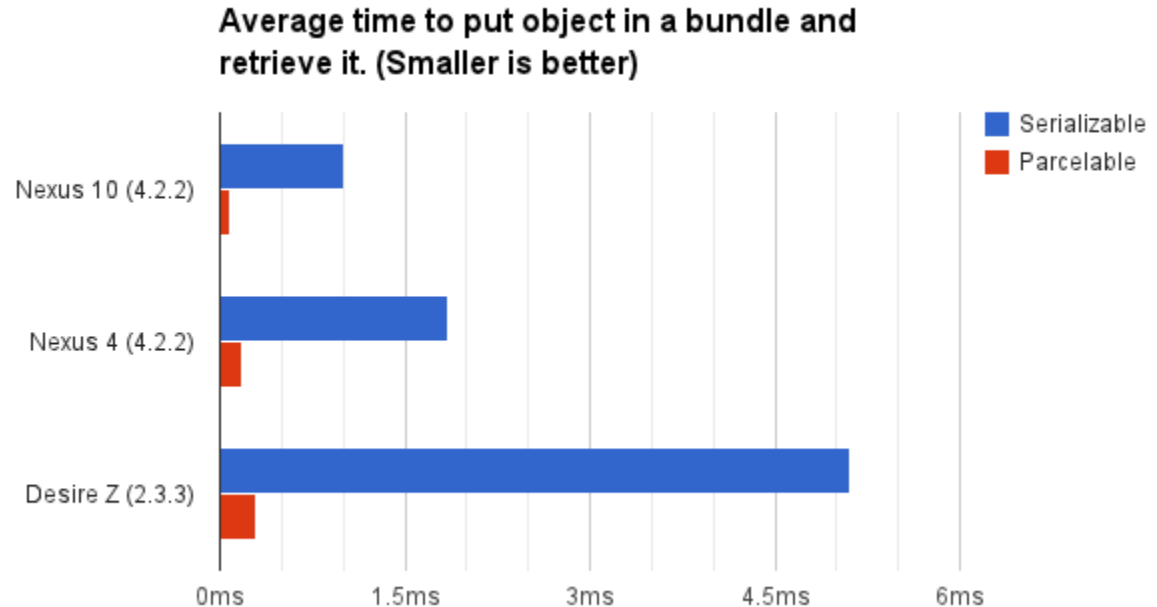
        public MyType[] newArray(int arg0) {
            MyType[] mta = new MyType[arg0];
            return mta;
        }
    };
}
```

- Kierunek komunikacji opisywany jest przez specjalne modyfikatory:
 - **in**: parametr przekazuje dane wejściowe dla serwisu (domyślna i jedyna opcja dla typów prymitywnych)
 - **out**: parametr służy za kontener, którego wartość zostanie nadpisana przez serwis
 - **inout**: parametr przekazuje dane wejściowe do serwisu, ale serwis może również modyfikować jego wartość



Parcelable vs Serializable

- **Parcelable** - implementujemy samodzielnie
- **Serializable** – automatyczna implementacja





W kliencie obowiązkowo należy przechwytywać wyjątki:

- RemoteException
- DeadObjectException
- TransactionTooLargeException

```
try {  
    double d = remote.getAzimuth();  
    List l = remote.getTrack(5);  
    Toast.makeText(MainActivity.this, d+" "+l.size(),  
        Toast.LENGTH_LONG).show();  
} catch (RemoteException e) {  
    e.printStackTrace();  
}
```

- Nie można rzucić wyjątku od serwisu do klienta



- Jeśli AIDL jest używany w obrębie jednego procesu, wówczas wątek żądania jest również wątkiem wykonania (tłumaczenie: bez sensu używać AIDL, wystarczy zaimplementować Binder)
- Jeśli następuje żądanie międzyprocesowe, nie ma kontroli nad tym, w którym wątku zostanie ono wykonane (np. mogą wystąpić równoczesne żądania w dwóch wątkach procesu roboczego, implementacja musi być thread-safe)
- Modyfikator *oneway*: powoduje że żądanie realizowane jest asynchronicznie



Mobilne systemy operacyjne: Programowanie natywne w systemie Android

Marek Kulawiak, Krzysztof Bruniecki

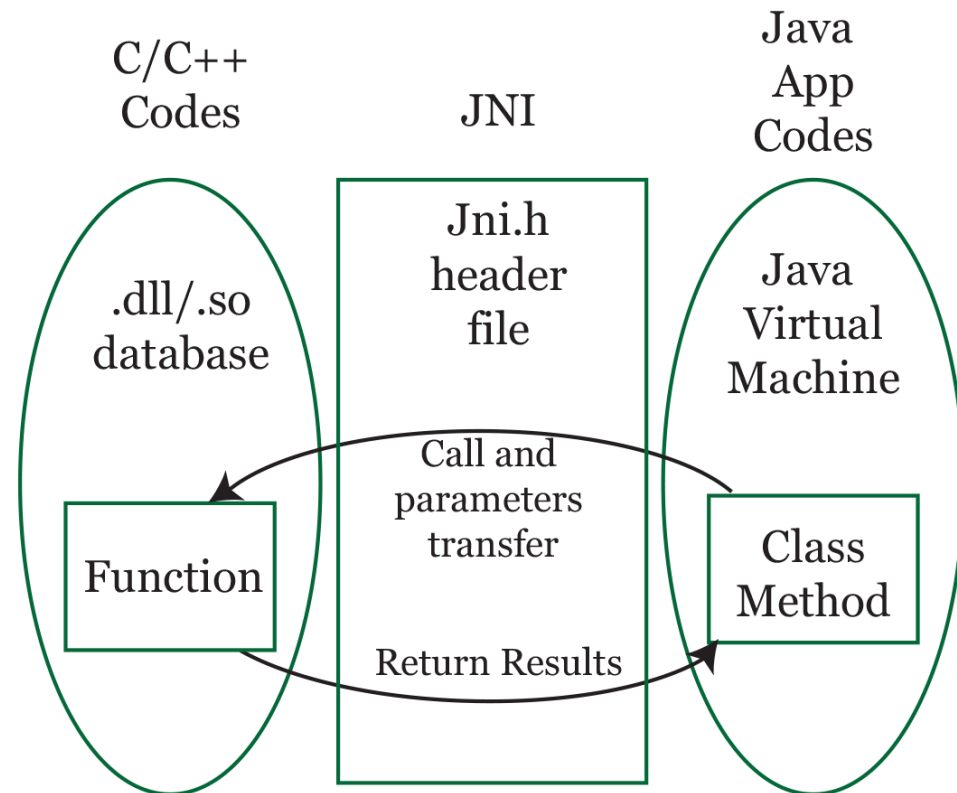


- Programowanie natywne polega na tworzeniu kodu kompilowanego dla konkretnej architektury procesora (np. ARM), zamiast pośrednictwa maszyny wirtualnej
- Na platformie Android do tworzenia kodu natywnego wykorzystuje się pakiet Android Native Development Kit (NDK)
- Możliwość programowania w C i C++
- Kod natywny kompilowany jest do postaci bibliotek współdzielonych, które umieszczane są w pliku *.apk w odrębnych podkatalogach dla różnych typów CPU (ABI)
 - np. *lib/armeabi/lib_costam.so*



Java Native Interface

- Standardowy interfejs pozwalający na komunikację pomiędzy kodem Javy uruchamianym wewnątrz maszyny wirtualnej (ART / Dalvik) a bibliotekami i aplikacjami stworzonymi za pomocą innych języków programowania (C, C++)
- Mapowanie definicji funkcji oraz typów zmiennych pomiędzy językami Java i C/C++ odbywa się za pośrednictwem pliku jni.h (występującego w wielu wariantach w zależności od wersji systemu i architektury urządzenia)





Zalety i wady programowania natywnego

- ✓ Możliwość wykorzystania gotowych bibliotek stworzonych w innym języku bez konieczności ich przepisywania
- ✓ Optymalizacja prędkości działania aplikacji poprzez bezpośrednie odwoływanie się do konkretnego sprzętu
- × Wzrost złożoności aplikacji
- × Trudniejsze debugowanie
- × Narzut komunikacyjny JNI
 - Konieczność (możliwość?) dostosowania pod różne platformy sprzętowe



Typy proste Java i ich natywne odpowiedniki

Java	Typ natywny	Rozmiar
boolean	jboolean	unsigned 8 bits
byte	jbyte	signed 8 bits
char	jchar	unsigned 16 bits
short	jshort	signed 16 bits
int	jint	signed 32 bits
long	jlong	signed 64 bits
float	jfloat	32 bits
double	jdouble	64 bits
void	void	N/A

Przykładowe definicje w pliku nagłówkowym *jni.h*:

```
typedef int jint;
```

```
#define JNI_TRUE 1
```



Typy złożone

- jobject
- jclass
- jstring
- throwable
- jarray
- jarray
- jobjectarray
- jbooleanarray
- jbytearray
- jchararray
- jshortarray
- jintarray
- jlongarray
- jfloatarray
- jdoublearray



```
android {  
    ...  
    defaultConfig {  
        ...  
        externalNativeBuild {  
            cmake {  
                cppFlags += "-std=c++17 -frtti -fexceptions"  
            }  
        }  
    }  
    ...  
    externalNativeBuild {  
        cmake {  
            path = file("src/main/cpp/CMakeLists.txt")  
            version = "3.22.1"  
        }  
    }  
    ...  
}
```




```
package ksg.mso.cpptest

class Calculator {
    external fun sum(x:Long, y:Long): Long

    companion object {
        init {
            System.loadLibrary("cpptest")
        }
    }
}
```




```
#include <jni.h>
#include <string>

extern "C"
JNIEXPORT jlong JNICALL
Java_ksg_mso_cpptest_Calculator_sum(
    JNIEnv *env, jobject thiz,
    jlong x, jlong y) {
    return x + y;
}
```



- Narzędzia do budowania bibliotek natywnych na podstawie kodu C / C++ i umieszczania ich wewnątrz plików APK
- Biblioteki natywne, takie jak:
 - libc, libm, libz, liblog, OpenSL ES
- Pliki nagłówkowe JNI oraz OpenGL ES
- Przykładowe projekty
- Dokumentacja



- Narzędzie do zarządzania procesem budowania projektów C/C++
- Nie kompiluje programu, a jedynie generuje pliki z regułami dla konkretnego środowiska, np.
 - pliki Makefile (Linux)
 - projekt Visual Studio (Windows)
- Wykorzystanie CMakeLists.txt przez Android Studio



- Android wykorzystuje własną wersję biblioteki C (autorstwa Google) o nazwie Bionic, dostosowaną dla urządzeń mobilnych
- Główne cechy Bionic:
 - Mało restrykcyjna licencja BSD, zezwalająca m.in. na statyczne linkowanie
 - Mniejszy rozmiar od biblioteki C GNU
 - Dostosowanie wyłącznie do procesorów wykorzystywanych w urządzeniach opartych na systemie Android



- STL jest to zestaw gotowych klas C++, udostępniających takie elementy języka jak listy, wektory czy iteratory
- Od czasu NDK w wersji 18, jedyną załączoną biblioteką STL jest libc++, ze względu na jej wsparcie dla nowszych standardów (C++11/C++14)
- Przykłady alternatywnych STL:
 - stlport, gnustl, gabi++



- Biblioteki C++ mogą być statyczne lub dynamiczne
- W przypadku libc++, programista ma do wyboru biblioteki *libc++_shared.so* oraz *libc++_static.a*
- Jeżeli dana aplikacja korzysta z więcej niż jednej biblioteki natywnej, zalecane jest stosowanie linkowania dynamicznego



- Wygenerowane pliki `.so` (biblioteki dynamiczne) oraz `.a` (biblioteki statyczne) nazywane są wg schematu:
 - *lib + nazwa_modułu + rozszerzenie*
- Jeżeli nazwa modułu zaczyna się od liter *lib*, wówczas przedrostek ten nie zostanie dodany do nazwy zbudowanej biblioteki.



- Kod natywny wygenerowany przez NDK dostosowany jest do konkretnego ABI (Application Binary Interface) opisującego sposób współpracy kodu maszynowego z systemem
- Typowe ABI zawiera szczegółowe informacje dotyczące konkretnych typów procesora



- Typowe ABI zawiera następujące informacje:
 - Zestaw instrukcji udostępnianych przez procesor
 - Kolejność bajtów (ang. endianness)
 - Format plików wykonywalnych
 - Konwencje dotyczące przekazywania danych pomiędzy kodem a systemem
 - Symbole funkcji dostępnych w czasie działania programu



- Platforma Android aktualnie wspiera następujące rodzaje ABI:
- armeabi (ARMV5TE, Thumb-1)
- armeabi-v7a (armeabi, Thumb-2, VFPv3-D16)
- arm64-v8a (AArch-64)
- x86 (x86, MMX, SSE/2/3, SSSE3)
- x86-64 (x86-64, MMX, SSE/2/3, SSSE3, SSE4.1/4.2, POPCNT)
- mips (MIPS32r1)
- mips 64 (MIPS64r6)



- W starszych wersjach NDK zamiast skryptów Cmake stosowano dwa rodzaje plików konfiguracyjnych .mk (makefile):
 - *Android.mk*
 - definiuje m.in. nazwy tworzonych modułów, ścieżki plików źródłowych oraz nazwy bibliotek wymaganych podczas linkowania
 - *Application.mk* (plik opcjonalny)
 - definiuje m.in. obsługiwane ABI oraz nazwę wybranej biblioteki STL (innej niż domyślna)



- Poniższy przykład stanowi instrukcję zbudowania biblioteki o nazwie **libCalculator.so**:

```
LOCAL_PATH := $(call my-dir)      #aktualna ścieżka  
include $(CLEAR_VARS)             #wyczyszczenie zmiennych  
LOCAL_MODULE := Calculator         #nazwa modułu  
LOCAL_SRC_FILES := ndk_mso_test_Calculator.cpp  
include $(BUILD_SHARED_LIBRARY)    #rodzaj projektu
```



- Zawartość typowego pliku Application.mk zwykle sprowadza się do nie więcej niż kilku linijek, np.:

```
APP_ABI := armeabi armeabi-v7a
```

- Powyższa linijka spowoduje wygenerowanie kodu maszynowego dla procesorów opartych na architekturze ARMv5TE oraz ARMv7. W przypadku chęci wspierania wszystkich dostępnych ABI, dla wygody można skorzystać z poniższego polecenia:

```
APP_ABI := all
```



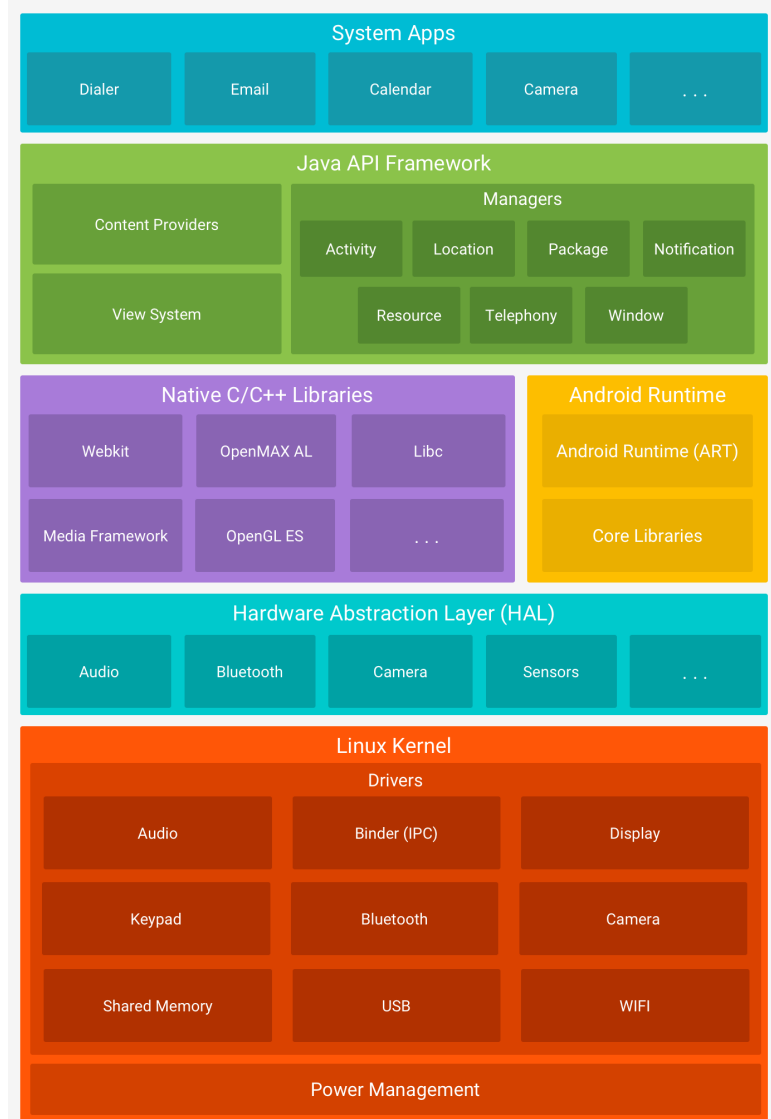
Mobilne systemy operacyjne: Architektura systemów Android i iOS

Krzysztof Bruniecki, Marek Kulawiak



Podział systemu operacyjnego Android

- Aplikacje użytkownika
- Zbiór API związanych z UI oraz usług systemowych
- Maszyna wirtualna (ART) i biblioteki natywne
- Warstwa HAL udostępniająca biblioteki do komunikacji ze sprzętem
- Jądro systemu (Kernel), zarządzające procesami i komunikujące się bezpośrednio ze sprzętem





- Jądro systemu oparte na kernelu Linuxa, odpowiedzialne za bezpośrednią komunikację z warstwą sprzętową
- Z punktu widzenia użytkownika systemu, wyróżnić można następujące bloki funkcjonalne jądra:
 - zarządzanie procesami, pamięcią i systemami plików
 - obsługa urządzeń
 - obsługa sieci
- Kernel zawiera mechanizmy bezpieczeństwa, takie jak:
 - możliwość nadawania uprawnień dostępu
 - izolację procesów



Jądro systemu Android zawiera szereg mechanizmów nie występujących w standardowym kernelu Linuxa, jak np.:

- Binder, wykorzystywany w komunikacji pomiędzy procesami (IPC)
- Logger, pozwalający na rejestrowanie różnych zdarzeń zachodzących w systemie
- własny podsystem zarządzania zużyciem energii (Power Management)
- obsługę systemu plików Yaffs (Yet Another Flash File System) w wersji 2



- Warstwa HAL udostępnia zestaw interfejsów pozwalających na wykorzystanie zasobów sprzętowych przez API Javy
- HAL składa się z wielu modułów, gdzie każdy moduł implementuje interfejs dla konkretnego typu komponentu sprzętowego (kamera, Bluetooth, itp.)
 - dany moduł jest wczytywany wtedy, gdy w ramach API wyższego poziomu wywoływany jest dostęp do danego komponentu sprzętowego
- HAL definiuje standardowy interfejs, który implementowany jest przez dostawców sprzętu, dzięki czemu wyższe warstwy systemu operacyjnego mogą działać niezależnie od sterowników sprzętowych

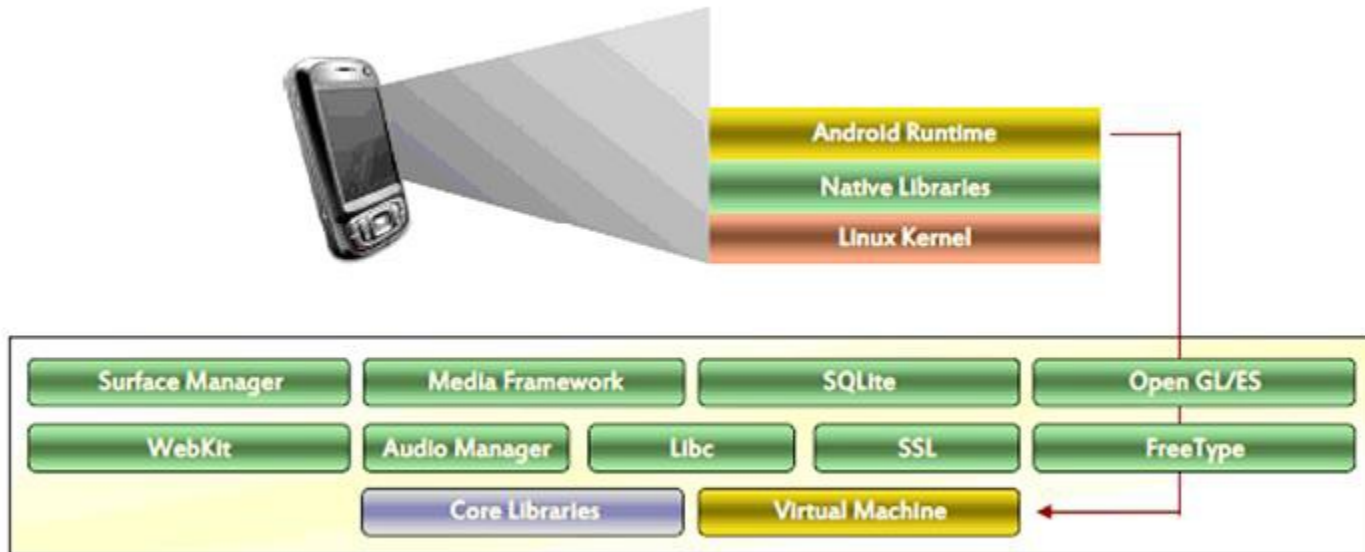


- Warstwa HAL z początku była silnie zintegrowana z wyższymi warstwami systemu
- Obecnie jest ona oddzielona:
 - komponenty HAL umieszczane są na osobnej partycji /vendor
 - komunikacja z komponentami HAL odbywa się za pomocą AIDL, który przejął rolę HIDL (HAL interface definition language)
- Powyższa separacja umożliwia podmianę frameworku Androida bez konieczności przebudowania HAL



Android Runtime (ART)

- Środowisko uruchomieniowe (proces) z przygotowaną maszyną wirtualną
- Maszyną wirtualną był niegdyś Dalvik, obecnie jest to ART (Android Runtime)
 - Logicznie oparta na rejestrach
 - Transkrypcja class -> dex na etapie kompilacji projektu





- Bionic: standardowa biblioteka języka C (libc.so)
 - malloc, atof, printf, itp.
 - mniejszy rozmiar od biblioteki C GNU
 - mało restrykcyjna licencja BSD
 - uproszczona pod kątem wolniejszych CPU
- Surface Manager
 - dokonuje kompozycji ekranów różnych aplikacji
 - używa akceleracji
 - pozwala łączyć 2D z 3D

- Biblioteki zapewniające abstrakcję (API) dla sprzętu różnego rodzaju

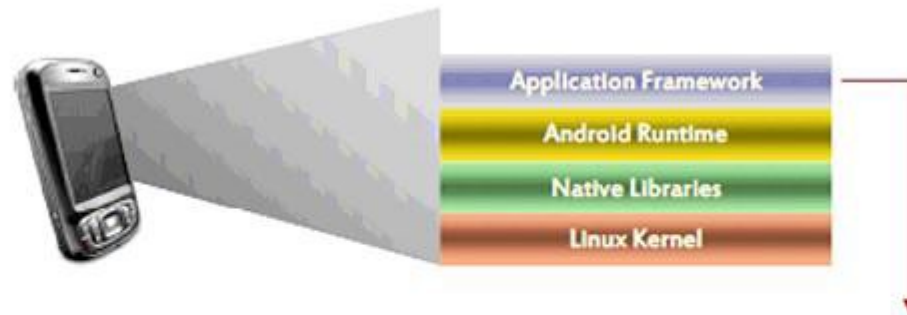


- Audio Manager





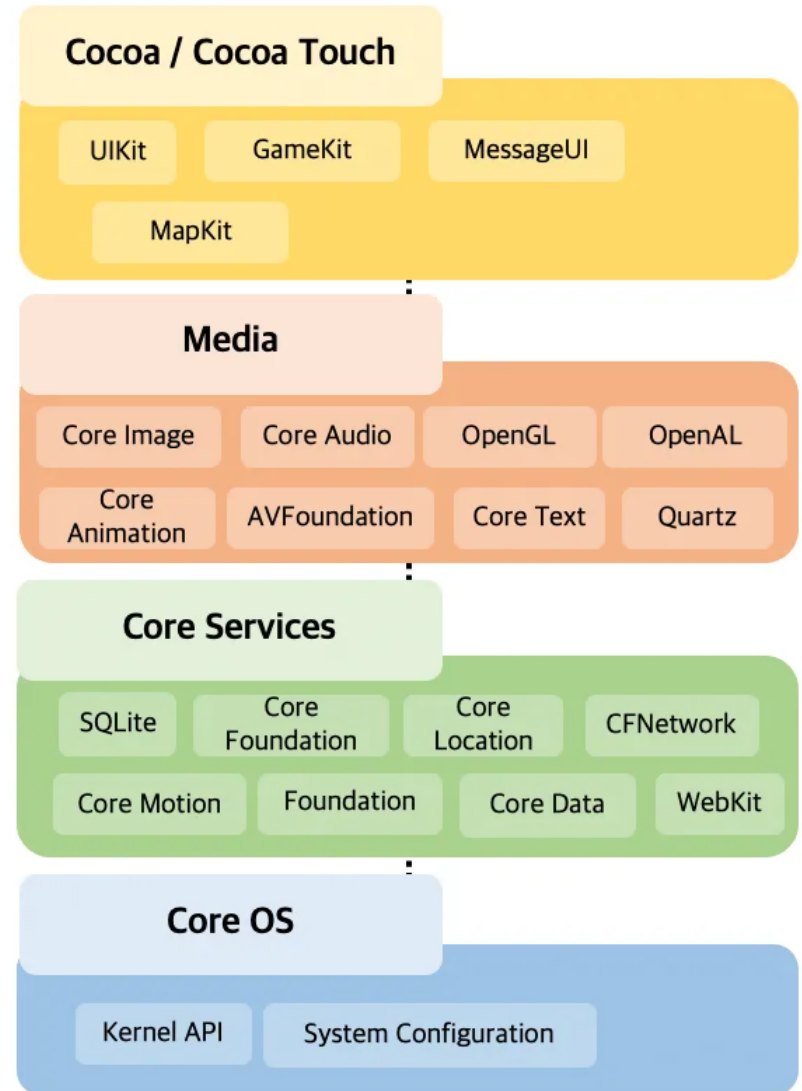
- System widoków (interfejs użytkownika)
- Zbiór niezależnych serwisów kontrolujących zasoby sprzętowe
 - Dostęp poprzez getSystemService()
 - LOCATION_SERVICE, NOTIFICATION_SERVICE, POWER_SERVICE, DOWNLOAD_SERVICE, VIBRATOR_MANAGER_SERVICE, ...





Podział systemu operacyjnego iOS

- Cocoa (Touch): zbiór API związanych z UI
- Media: zestaw frameworków do grafiki i dźwięku
- Core Services: zbiór serwisów niezwiązanych z UI
- Core OS: jądro systemu i sterowniki





Darwin – podstawa systemów Apple

iOS

Cocoa Touch

Media

Core Services

Core OS

Darwin OS
(Open Source)

XNU
Hybrid Kernel

Networking

BSD

Filesystem

Drivers

Mach Microkernel

I/O Kit



**Mobilne systemy operacyjne:
Maszyny wirtualne w systemach mobilnych**

Krzysztof Bruniecki, Marek Kulawiak



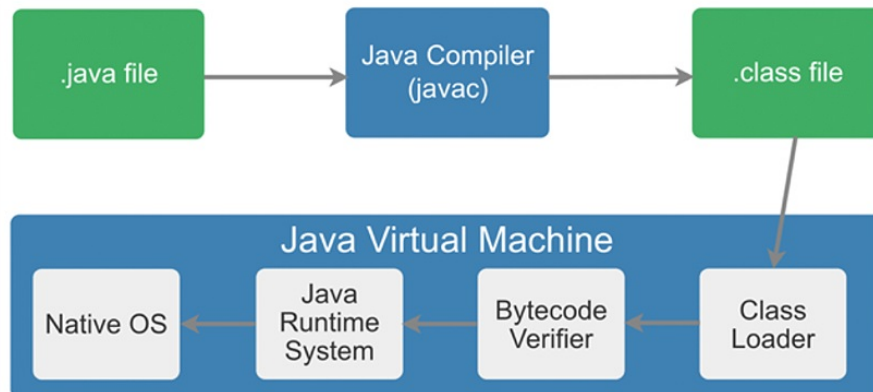
Wyróżnia się dwa rodzaje maszyn wirtualnych:

- takie, które pozwalają emulować daną platformę sprzętową i uruchamiać pełne systemy operacyjne (*system virtual machines*)
- takie, które pozwalają uruchamiać programy komputerowe działające w sposób niezależny od platformy sprzętowej i systemu operacyjnego (*process virtual machines*)



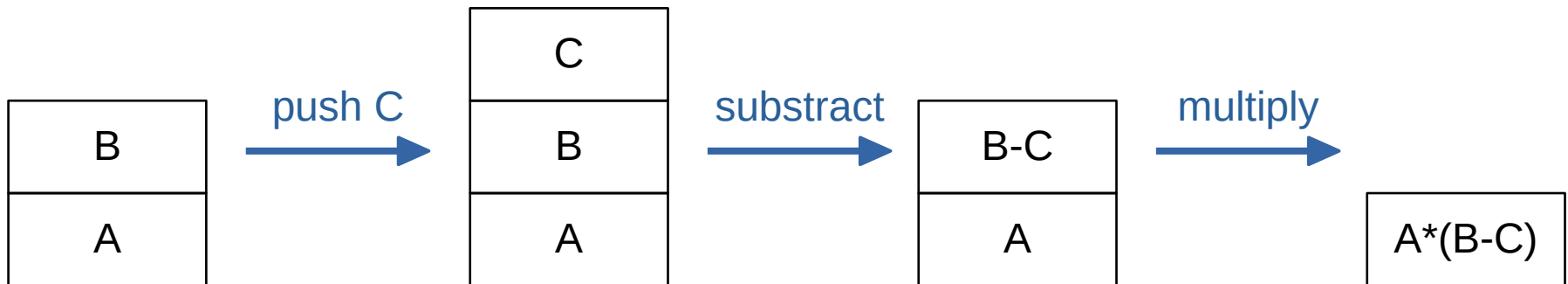
Przykładem maszyny drugiego rodzaju jest Java Virtual Machine:

- najpierw kod źródłowy programu zostaje przetworzony do postaci kodu bajtowego Javy
- następnie kod bajtowy jest tłumaczony do postaci kodu maszynowego właściwego dla danej platformy sprzętowej



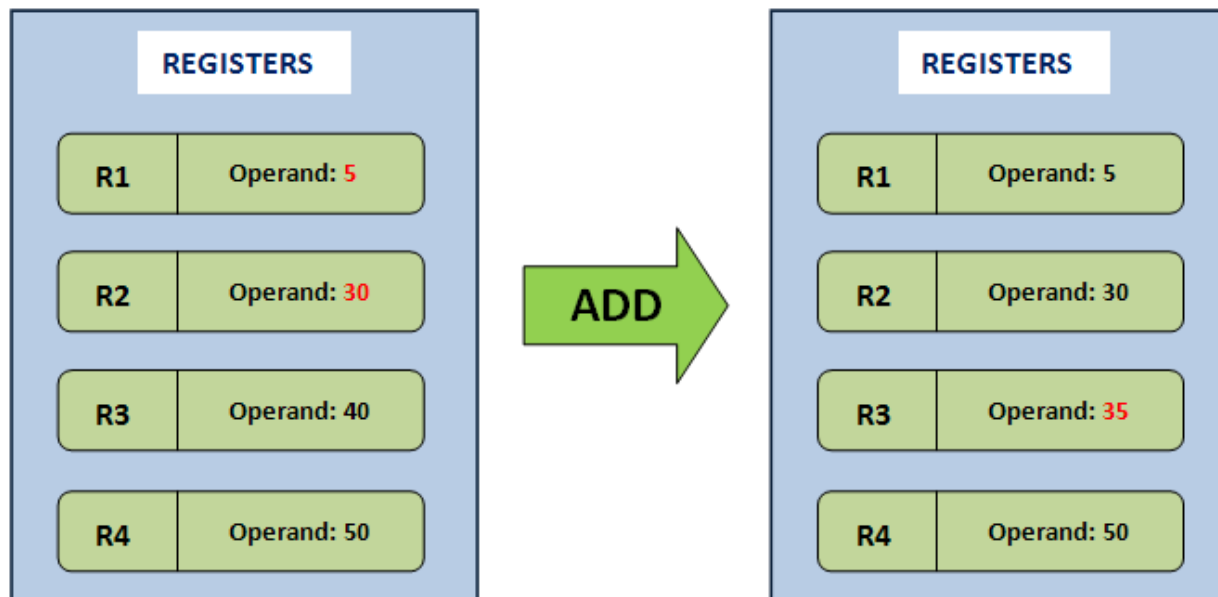


- JVM jest przykładem maszyny opartej na stosie
 - dane są w odpowiedniej kolejności umieszczane na stosie oraz przetwarzane za pomocą krótkich instrukcji





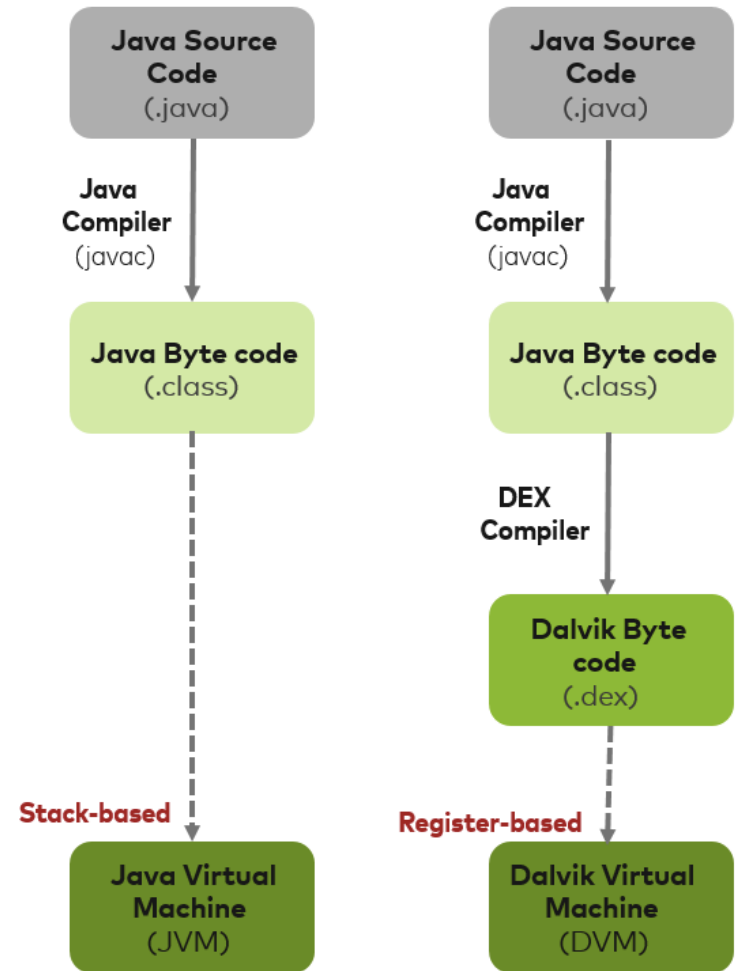
- Alternatywą dla stosu są rejestry przechowujące poszczególne wartości
 - przykład: ADD R1, R2, R3 ;





Dalvik Virtual Machine (DVM)

- Stworzona na potrzeby Androida maszyna wirtualna Dalvik oparta jest na rejestrach
- Kod źródłowy aplikacji przetwarzany jest najpierw do postaci kodu bajtowego Javy, a następnie do formatu kodu bajtowego .dex (Dalvik EXecutable)
- Od Androida 5.0 rolę Dalvika przejął Android Runtime (ART), który również korzysta z plików .dex





- ART zawiera szereg usprawnień w stosunku do Dalvika:
 - kompilacja AOT (ahead-of-time), gdzie kod .dex konwertowany jest do kodu maszynowego na etapie instalacji aplikacji
 - zoptymalizowany Garbage Collector
 - usprawniony debugging
 - szybszy czas działania = mniejsza energochłonność
- Wadą ART jest wydłużenie czasu instalacji aplikacji z powodu kompilacji AOT, a także zwiększenie rozmiaru zainstalowanych aplikacji poprzez konieczność przechowywania kodu maszynowego



- Przykład: "**move-wide/from16 vAA, vBBBB**":
 - **move** – rodzaj bazowej operacji (przeniesienie wartości pomiędzy rejestrami)
 - **wide** – przyrostek oznaczający operowanie na danych 64-bitowych
 - **from16** – źródłem jest rejestr, którego numer jest zapisany na 16 bitach
 - **vAA** – rejestr docelowy, zakres v0 – v255 (8 bitów)
 - **vBBBB** – rejestr źródłowy, zakres v0 – v65535 (16 bitów)



Bajtkod Dalvika – przykład (1)

```
public class Testowa {
```

```
    public int pole;
```

```
    public Testowa() { ...
```

Class #0 -

Class descriptor : 'LTestowa;'

Access flags : 0x0001 (PUBLIC)

Superclass : 'Ljava/lang/Object;'

Interfaces -

Static fields -

Instance fields -

#0 : (in LTestowa;)

name : 'pole'

type : 'I'

access : 0x0001 (PUBLIC)

Direct methods -

#0 : (in LTestowa;)

name : '<init>'

type : '()V'

access : 0x10001 (PUBLIC CONSTRUCTOR)

...



Bajtkod Dalvika – przykład (2)

```
public int funkcja_abs() {  
    if (pole > 0) {  
        return pole;  
    }  
    else {  
        return -pole;  
    }  
}
```

```
[000110] Testowa.funkcja_abs:()I  
0000: iget v0, v1, LTestowa;.pole:I  
0002: if-lez v0, 0007  
0004: iget v0, v1, LTestowa;.pole:I  
0006: return v0  
0007: iget v0, v1, LTestowa;.pole:I  
0009: neg-int v0, v0  
000a: goto 0006
```



Getting root access on Android devices

Marcin Kulawiak

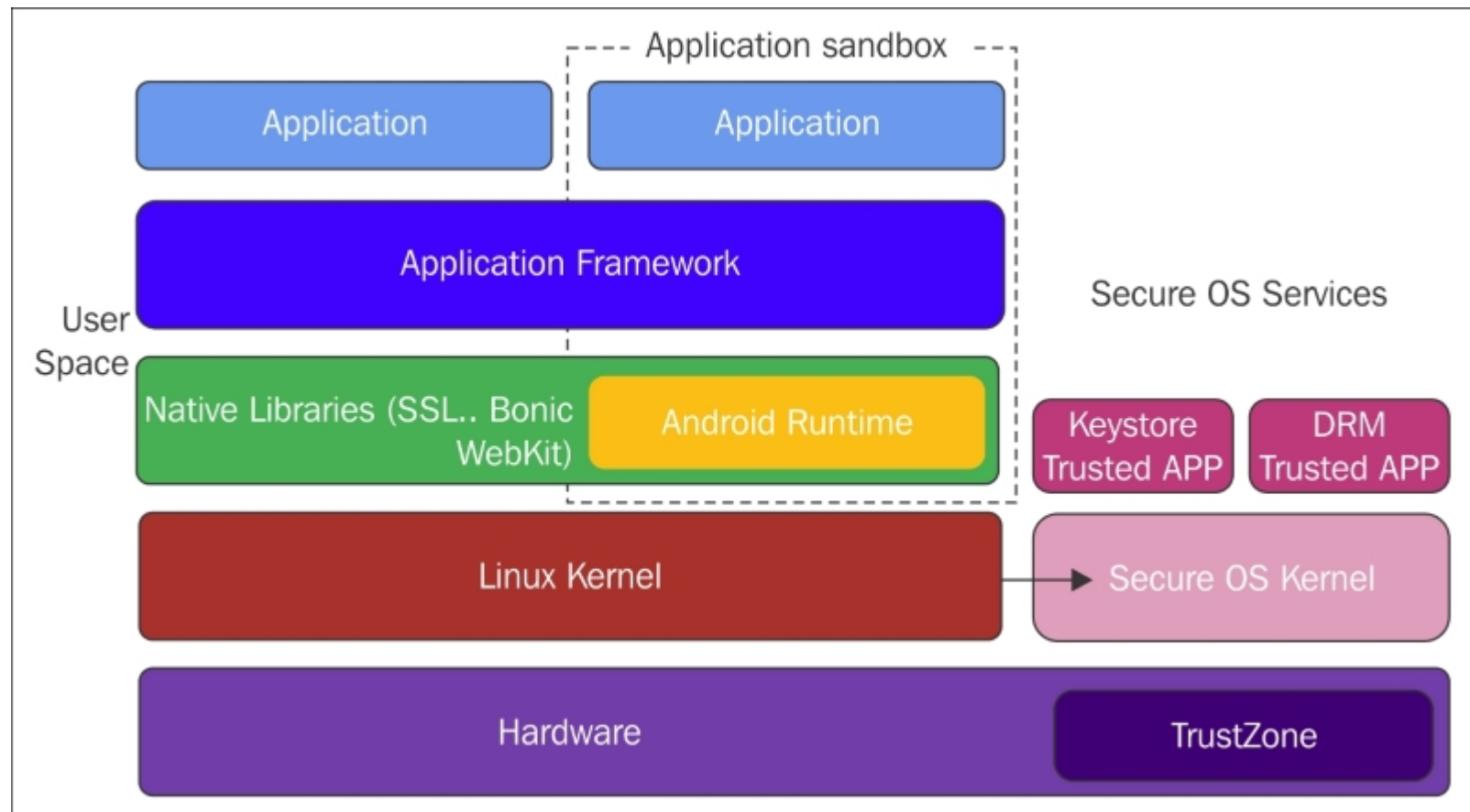
Katedra Systemów Geoinformatycznych

Wydział Elektroniki, Telekomunikacji i Informatyki Politechniki
Gdańskiej



Android security

By default, android apps work in a ‘sandbox’ mode





Android security

Android app sandbox is based on Security Enhanced Linux (SELinux). It features:

- system and app data/memory separation (since Android 5/6)
- limitation on available syscalls (since Android 8)
- individual sandboxes for all apps (since Android 9)
- file system protection via Storage Access Framework (since Android 10) (25–50x slower!)

Source: <https://source.android.com/security/app-sandbox>

Source: <https://www.xda-developers.com/android-q-storage-access-framework-scoped-storage/>



The superuser account

By default, Android contains several user accounts with different privilege levels:

- User (can only access dedicated filesystem space, can install apps which run in sandboxes)
- System (hosts all system services, can access filesystem and memory forbidden to regular users)
- Superuser (can do everything and anything, not accessible by default).



Accessing Superuser

Initially, accessing superuser privileges was very easy. However, with every Android revision the process became more and more complex.

First attempts at obtaining root privileges involved exploiting system vulnerabilities in order to gain temporal access to the root filesystem. This access was then used to upload the 'su' binary to '/system/bin/su' and grant it appropriate permissions.



Accessing Superuser

This approach worked for virtually all devices running Android 2.0–5.1.1, as long as the device's system exposed one of the known code vulnerabilities.

As a result, several scripts and applications using these universal rooting methods have been created, including Framaroot, Root_with_Restore, KingoRoot, OneClickRoot (commercial) and Root Master.



Accessing Superuser

Android 6.0 introduced checksum verification of the ‘/system’ partition at boot, which meant that it was no longer possible to achieve root access without unlocking the device’s bootloader.

While initially many manufacturers allowed for unlocking the bootloaders of their devices, most have since changed their policies. Examples of such manufacturers are Samsung and Huawei, who locked down their devices once they became hugely popular.



Accessing Superuser

Thus, accessing root on Android 6.0+ requires first unlocking the bootloader. For some manufacturers (Sony, LG, Xiaomi) the process is rather straightforward and requires registering at a dedicated web page, agreeing to void the device's warranty and obtaining an unlock code.

If the manufacturer does not provide this functionality, the bootloader must be unlocked via exploiting a known code vulnerability. While these are usually device-specific, a semi-universal tool exists in the form of CF-Auto-Root.



Unlocking bootloader

Regardless of the chosen unlock method, it is necessary to access the device's 'Developer Options' menu and enable 'OEM unlock'. If 'OEM unlock' is not available, it will most likely be impossible to unlock the bootloader. On the other hand, the existence of this option does not guarantee success.

Performing a successful 'OEM unlock' is equivalent to doing a 'Factory reset', which means that all the user's files and settings are wiped from the device.



Unlocking bootloader

Once the bootloader has been unlocked, unsigned code may be safely flashed to the boot partition. Beforehand, flashing such code would prevent the system from booting, causing a 'soft brick'.

The simplest modification involves patching the kernel image (boot.img) to inject the su daemon service and modified SELinux policies.

Such operations require Fastboot and ADB, which are usually installed as part of Android SDK. However, they can also be downloaded individually from:

<https://dl.google.com/android/repository/platform-tools-latest-windows.zip> [//'-linux.zip](https://dl.google.com/android/repository/platform-tools-latest-linux.zip), ['-darwin.zip](https://dl.google.com/android/repository/platform-tools-latest-darwin.zip)'
for Mac



Accessing Superuser

Simplest way to root after unlocking the bootloader
(requires ADB & fastboot drivers):

1. Download latest official ROM and extract boot.img, copy it to the device.
2. Install Magisk (previously Magisk Manager) apk and use it to patch the extracted boot.img on the device. Copy magisk_patch.img back PC.
3. Enter fastboot mode (usually by holding Volume Down and Power buttons during boot).
4. Using latest fastboot, flash the rooted kernel:
fastboot flash boot magisk_patch.img
fastboot reboot



Types of SU binaries

Although in general anyone can compile their own su binary for Android, over the years there have been many popular applications which packed the su binary together with an app for controlling root access permissions. Such apps include:

- Magisk – most recent, can be extended with modules (Xposed, Vanced, Hide), systemless.
- SuperSU – older and simpler, only provides root access control.
- SuperUser – one of the first su apps, discontinued.



System vs systemless root

With an unlocked bootloader it is possible to use both 'system' and 'systemless' approaches (using root on a locked bootloader on 5.1.1 and earlier versions required a modified '/system'). Modification of '/system' is practically equivalent to flashing a custom ROM – the OS is no longer considered 'official'. A systemless approach means that the ROM remains unchanged, and it is possible eg. to perform an incremental upgrade to a new version of an 'official' ROM using a flashable zip (Over The Air update – 'OTA').

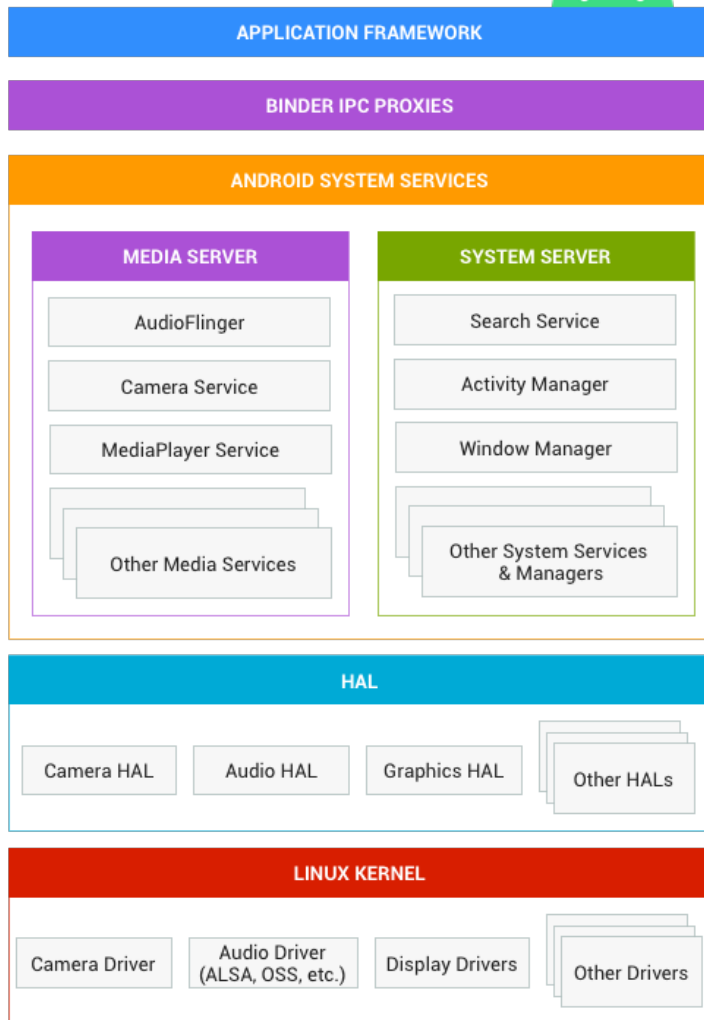


Android distributions with built-in root

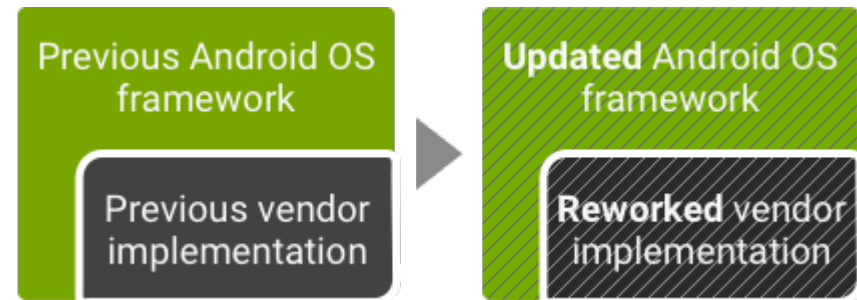
With an unlocked bootloader, it is possible to flash a custom Android ROM. Custom ROMs consist of a kernel image (which needs to be flashed first) and a system image (which contains the root filesystem). Historically, a custom ROM image needed to be pre-built for a given device (the device drivers needed to be compiled into the kernel). Devices which launched with Android 8 and later must support an Android Hardware Abstraction Layer described in HIDL, which allows for such devices to use any distribution of Android.



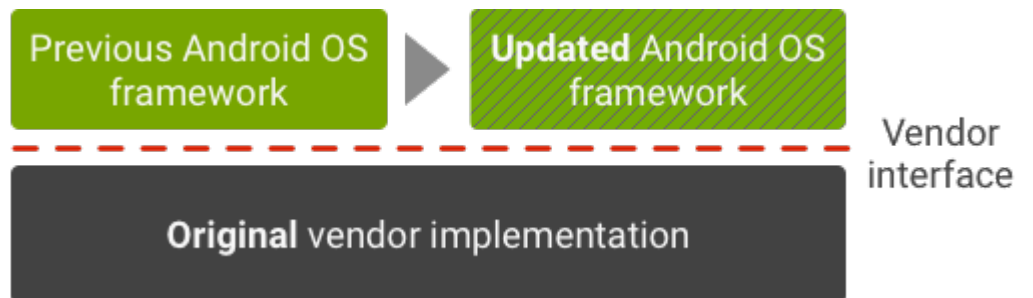
Android Hardware Abstraction Layer (HAL)



Before Android 8 HAL:



After:



Android distributions with built-in root

Popular custom ROMs with built-in root support include:

LineageOS – contains the largest number of custom features, comes pre-built for the largest number of devices. Successor to CyanogenMOD.

CyanogenMOD – a hugely modified version of AOSP, came pre-built for hundreds of devices. Discontinued.

AOKP – Android Open Kang Project. Initially forked from AOSP (Android Open Source Project), now based on LineageOS.

NAOSP – Near AOSP. A name commonly used for customized AOSP builds which usually come with root.



Custom recovery

For convenience and increased safety, after unlocking bootloader it is recommended to flash a custom recovery such as TWRP. A custom recovery enables on-device system and data backup (eg. to an external SD card) and allows for flashing custom ROMs directly from the device. It can also be used for formatting and repartitioning the internal SD card (although the latter is not recommended due to the current structure of Android – A/B Seamless Updates require the existence of certain partitions, for example).



Flashing a custom recovery

1. Install ADB and Fastboot drivers for your device.
2. Obtain a custom recovery for your device (eg. Team Win Recovery Project – <https://twrp.me>).
3. Enable USB debugging in ‘Developer Options’.
4. Reboot device into bootloader
`adb reboot bootloader`
5. Flash chosen recovery (substitute ‘twrp’ with the name of your recovery file):
`fastboot flash recovery twrp.img`
6. Reboot. Recovery can now be accessed at boot.



Issues with rooting

Because rooting now requires unlocking the bootloader, a rooted device will fail the SafetyNet Hardware Attestation check (the certificate chain from hardware to system will be broken), and as a result it will be recognized as unsafe by Google Services.

Rooted applications are banned from using many payment services (eg. Google Pay) and lose support for Widevine DRM (which means Netflix only in 720p).

Moreover, apps with root access can potentially leak valuable data or even damage the device hardware. Proceed with caution.



Recommended reading

<https://www.xda-developers.com/install-adb-windows-macos-linux/>

<https://www.xda-developers.com/how-to-install-twrp/>

<https://www.xda-developers.com/how-to-root-your-android-without-a-computer/>

<https://www.xda-developers.com/bypass-safetynet-hardware-attestation-unlocked-bootloader-magisk-module/>



POLITECHNIKA
GDAŃSKA

Thank You!